
CSS Básico: El Diseño que Transforma la Web

Una Novela para Aprender CSS Desde Cero

Alex Goyzueta Delgado
alexgoyzueta2018@gmail.com

CSS BÁSICO

El Diseño que Transforma la Web

Una Novela para Aprender CSS Desde Cero

Autor: Alex Goyzueta Delgado

Contacto: alexgoyzueta2018@gmail.com

> *"El diseño no es solo cómo se ve,*

> *sino cómo funciona.*

> *En cada regla CSS hay una decisión que transforma la experiencia."*

Lima, 2026

Créditos

CSS Básico: El Diseño que Transforma la Web

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados. Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en forma alguna ni por ningún medio electrónico, mecánico, fotocopia, grabación u otro método, sin el permiso previo por escrito del autor.

Edición y corrección: Alex Goyzueta Delgado

Ilustraciones conceptuales: Generadas mediante descripciones textuales

Diseño de portada: Alex Goyzueta Delgado

Tipografía: IBM Plex Mono (código), Lato (narrativa)

Agradecimientos especiales:

A los diseñadores y desarrolladores que construyen la web con pasión y creatividad.

A los que enseñan con paciencia, como Carolina le enseñó a Mateo.

A todos los que creen que el diseño puede cambiar el mundo.

Datos de catalogación:

Goyzueta Delgado, Alex

CSS Básico: El Diseño que Transforma la Web / Alex Goyzueta Delgado

1ra edición — Lima, 2026

Dedicatoria

A los diseñadores que ven belleza en el caos
y encuentran orden en el código.

A los desarrolladores novatos que escriben su primer selector
con la misma emoción con que un pintor carga su pincel por primera vez.

A los que creen que una página web puede ser más que texto sobre blanco:
una experiencia, un viaje, una obra de arte.

"El diseño no es decoración. Es comunicación.

Cada color, cada espacio, cada tipografía dice algo."

— Carolina, mentora en PixelLab

Prefacio

¿Por qué este libro?

CSS (Cascading Style Sheets) es el lenguaje que da vida visual a la web. Sin CSS, internet sería un amasijo de texto negro sobre fondo blanco, sin colores, sin diseños, sin personalidad. Pero aprender CSS puede ser abrumador: selectores, modelo de caja, flexbox, grid, animaciones... ¿por dónde empezar?

Este libro nació de una convicción: **la mejor manera de aprender CSS es necesitarlo de verdad.**

No vas a memorizar propiedades de memoria. Vas a acompañar a **Mateo Vargas**, un desarrollador novato que acaba de conseguir trabajo en PixelLab, una agencia digital, y debe convertir el sitio HTML de la revista "Cultura Viva" en algo visualmente impresionante. En el proceso, aprenderás CSS desde cero mientras ayudas a Mateo a salvar su primer proyecto profesional.

¿Cómo usar este libro?

Cada capítulo técnico (del 1 al 10) tiene tres partes:

1. **Narrativa** — La historia avanza. Mateo enfrenta desafíos de diseño, comete errores, aprende de Carolina y crece como desarrollador.
2. **Explicación técnica** — Los conceptos de CSS que Mateo usa para resolver sus problemas. Explicados con claridad, con ejemplos prácticos.
3. **Enigmas** — Ejercicios para que tú mismo practiques lo aprendido. Al final del libro encontrarás las soluciones.

¿Qué necesitas?

- Un editor de texto (VS Code, Sublime Text, o cualquier otro)
- Un navegador web moderno (Chrome, Firefox, Edge)
- Conocimientos básicos de HTML (etiquetas, atributos, estructura de documento)
- Curiosidad y ganas de aprender

Un viaje, no un destino

Al terminar este libro, no solo sabrás cómo Mateo logró que "Cultura Viva" tuviera una portada espectacular. También habrás aprendido CSS desde los selectores básicos hasta animaciones, pasando por Flexbox, Grid, diseño responsivo y mucho más.

Y quizás descubras que CSS no es solo una herramienta técnica. Es un lienzo donde pintar ideas, un medio para contar historias visuales, una habilidad que transforma la manera en que ves la web.

— *Alex Goyzueta Delgado*

Lima, 2026

Introducción

Lima, 2026

Mateo Vargas tenía 24 años, un título en diseño gráfico que apenas usaba, y una cuenta bancaria que rozaba el cero absoluto. Llevaba seis meses enviando currículos a toda agencia digital de Lima que tuviera una oferta de trabajo publicada. La mayoría ni siquiera le respondía.

Hasta que un jueves por la tarde, cuando ya empezaba a considerar seriamente la idea de mudarse de vuelta a casa de su mamá, su teléfono vibró con un correo que cambiaría su vida.

Asunto: Oferta de trabajo — Desarrollador Web Junior

Mateo casi deja caer el teléfono. Abrió el correo con manos temblorosas.

"Hola Mateo,

Hemos visto tu portafolio y creemos que tienes potencial. Buscamos a alguien con conocimientos de HTML y disposición para aprender CSS sobre la marcha. Nos encantaría conocerte en una entrevista.

— Carolina Rivas, Directora de Diseño en PixelLab"

PixelLab

PixelLab era una agencia digital de tamaño medio ubicada en Miraflores. Tenía quince empleados, una reputación sólida y clientes que iban desde startups tecnológicas hasta revistas culturales tradicionales.

La entrevista fue más una conversación que un interrogatorio. Carolina, una mujer de unos 35 años, de mirada aguda y sonrisa paciente, le mostró proyectos reales de la agencia.

—Sabes HTML, ¿verdad? —le preguntó Carolina.

—Sí —respondió Mateo con más seguridad de la que sentía—. Etiquetas, atributos, formularios, tablas. Lo básico.

—Perfecto. Porque tengo un proyecto para ti. La revista "Cultura Viva" quiere renovar su sitio web. Tienen el contenido en HTML, pero se ve... —Carolina buscó la palabra adecuada—. De los años noventa.

En la pantalla apareció el sitio de "Cultura Viva". Mateo intentó no hacer una mueca, pero falló. El sitio tenía texto negro sobre fondo blanco, imágenes sin bordes, enlaces en azul subrayado, y una distribución que parecía hecha por alguien que odiaba el diseño.

—Exactamente —rió Carolina—. Por eso necesitamos a alguien que lo haga ver bien. Tu trabajo será aplicar CSS para transformar este HTML en un sitio moderno y atractivo. ¿Crees que puedes hacerlo?

Mateo tragó saliva. Había hecho algunos tutoriales de CSS, pero nada serio. Sin embargo, necesitaba este trabajo.

—Sí —dijo—. Puedo hacerlo.
—Bienvenido a PixelLab, Mateo.

Lo que está en juego

El sitio de "Cultura Viva" era el proyecto insignia de la agencia. Si Mateo lograba transformarlo, PixelLab aseguraba un contrato anual con la revista. Si fallaba... bueno, Mateo prefería no pensar en esa posibilidad.

Tenía dos meses para aprender CSS y aplicarlo a un sitio web real. Su mentora sería Carolina, una diseñadora con diez años de experiencia que conocía CSS como la palma de su mano.

El primer día, Carolina le dio un consejo que Mateo nunca olvidaría:

—CSS no es difícil, Mateo. Es diferente. No pienses en términos de lógica de programación. Piensa en términos de diseño. Cada propiedad CSS es una decisión visual. ¿Qué quieres que el usuario vea primero? ¿Cómo quieres que se sienta al navegar? El código viene después.

Mateo asintió, abrió su editor de texto, y miró el código HTML de "Cultura Viva". Había docenas de archivos, cientos de líneas de código. Todo en HTML puro. Sin una sola línea de CSS.

—Por donde empiezo —murmuró.

—Por el principio —respondió Carolina, que lo estaba observando desde la puerta—. Por el primer estilo.

¿Qué aprenderás en este libro?

Acompaña a Mateo en su viaje. A través de diez capítulos, aprenderás:

1	Introducción, selectores, propiedades de CSS	El CSS para dar estilo a cualquier elemento
3	Modelo de caja (box model)	Entender cómo se distribuye el espacio
5	Flexbox	Layouts flexibles y modernos sin esfuerzo
7	Responsive Design y Media Queries	Sitios que se ven bien en cualquier pantalla
9	Pseudo-clases y pseudo-elementos	Detalles finos que marcan la diferencia

Cada capítulo incluye ejercicios prácticos para que practiques lo aprendido. Al final del libro encontrarás las soluciones.

La revista "Cultura Viva" te espera. ¿Estás listo para transformarla?

Capítulo 1: El Primer Estilo

Tema: Introducción a CSS, selectores básicos, propiedades de texto

Mateo llegó temprano a PixelLab. Eran las 8:30 de la mañana y la oficina aún estaba en silencio, iluminada por la luz grisácea del invierno limeño que entraba por los ventanales del tercer piso. Dejó su mochila en el escritorio que le habían asignado —una esquina modesta con un monitor Dell y una planta de plástico que parecía tan nerviosa como él— y abrió el editor de texto.

Carolina apareció media hora después con un café en una mano y una laptop en la otra.

—¿Listo para tu primera lección? —preguntó, sentándose a su lado.

Mateo asintió.

—Primero, veamos qué estamos construyendo —dijo Carolina, abriendo el archivo `index.html` de "Cultura Viva".

La pantalla mostró el código HTML de la portada de la revista. Había encabezados, párrafos, imágenes, enlaces. Todo perfectamente estructurado, pero visualmente plano.

—El HTML es el esqueleto —explicó Carolina—. Define qué es cada cosa: un título, un párrafo, una imagen. Pero no dice cómo se ve. Eso es trabajo de CSS.

¿Qué es CSS?

—CSS significa **Cascading Style Sheets** —dijo Carolina, escribiendo en un documento nuevo—. En español: Hojas de Estilo en Cascada. Es el lenguaje que usamos para describir la presentación visual de un documento HTML.

—¿Y lo de "en cascada"? —preguntó Mateo.

—Ya veremos. Por ahora, piensa en CSS como la pintura, los muebles y la decoración de una casa. El HTML son las paredes y el techo. Con CSS decides de qué color pintar, dónde poner los muebles, qué tipo de cortinas usar.

Formas de incluir CSS

Carolina abrió tres pestañas en el navegador para mostrar los métodos:

1. CSS en línea (inline)

```
<p style="color: blue; font-size: 18px;">Este párrafo es azul y grande.</p>
```

—Se aplica directamente a un elemento con el atributo `style` —explicó Carolina—. Es rápido, pero desordenado. Mezclas contenido con presentación.

2. CSS interno

```

<head>
  <style>
    p {
      color: blue;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <p>Este párrafo es azul y grande.</p>
</body>

```

—Se escribe dentro de la etiqueta ``<style>`` en el ``<head>`` del documento. Mejor que inline, pero solo funciona para una página.

3. CSS externo (el más recomendado)

```

/* estilo.css */
p {
  color: blue;
  font-size: 18px;
}

```

```

<head>
  <link rel="stylesheet" href="estilo.css">
</head>

```

—Un archivo separado con extensión `.css` que enlazamos desde el HTML con ``<link>``. Así mantenemos el estilo separado del contenido y podemos reutilizarlo en muchas páginas.

—Este es el método que usaremos en "Cultura Viva" —dijo Carolina—. Un solo archivo CSS para todo el sitio.

Sintaxis básica de CSS

Carolina abrió un archivo nuevo y escribió:

```

selector {
  propiedad: valor;
  propiedad: valor;
}

```

—Cada regla CSS tiene tres partes: un **selector** (¿a qué elementos aplicamos el estilo?), una **propiedad** (¿qué aspecto queremos cambiar?) y un **valor** (¿cómo queremos que sea?).

—Por ejemplo:

```

h1 {
  color: darkred;
  font-size: 36px;
  text-align: center;
}

```

—Esto dice: "todos los encabezados `h1` deben ser de color rojo oscuro, tamaño 36 píxeles, y centrados".

Selectores básicos

—Ahora, lo más importante: los **selectores** —dijo Carolina, abriendo el HTML de "Cultura Viva" en el navegador—. Necesitas una forma de apuntar a elementos específicos. Hay tres selectores básicos que usarás todo el tiempo:

Selector de tipo (etiqueta)

```
/* Afecta a TODOS los elementos de ese tipo */
p {
  font-family: Georgia, serif;
  line-height: 1.6;
}

h2 {
  color: #333;
}
```

Selector de clase

En HTML:

```
<p class="destacado">Este párrafo especial necesita destacar.</p>
```

En CSS:

```
/* Las clases comienzan con un punto (.) */
.destacado {
  background-color: #ffffcc;
  font-weight: bold;
  padding: 10px;
}
```

—Las **clases** son reutilizables. Puedes aplicar la misma clase a muchos elementos diferentes.

Selector de ID

En HTML:

```
<h1 id="titulo-principal">Cultura Viva</h1>
```

En CSS:

```
/* Los ID comienzan con numeral (#) */
#titulo-principal {
  color: #8B0000;
  font-size: 48px;
  text-transform: uppercase;
}
```

—Los **ID** deben ser únicos. Solo un elemento por página puede tener un ID específico.

Carolina mostró una tabla comparativa:

Selector	Sintaxis	Uso	Reutilizable
Tipo	`p { }`	Todos los ` <p>`</p>	Sí

Clase	<code>.clase { }</code>	Múltiples elementos	Sí
ID	<code>#id { }</code>	Un solo elemento	No

Propiedades de texto básicas

—Ahora, lo que más necesitarás para empezar: propiedades para dar estilo al texto.

Carolina escribió un ejemplo completo:

```
body {
  font-family: Arial, Helvetica, sans-serif;
  font-size: 16px;
  color: #333333;
}

h1 {
  font-size: 32px;
  font-weight: bold;
  color: #1a1a1a;
  text-align: center;
}

p {
  font-size: 16px;
  line-height: 1.5;
  text-align: justify;
}

.destacado {
  color: #cc0000;
  font-weight: bold;
  font-size: 18px;
}
```

Propiedades explicadas:

Propiedad	¿Qué hace?	Valores comunes
<code>color</code>	Color del texto	nombres, hex, rgb
<code>font-size</code>	Tamaño de letra	px, em, rem
<code>font-family</code>	Tipo de letra	Arial, Georgia, serif, sans-serif
<code>font-weight</code>	Grosor	normal, bold, 100-900
<code>text-align</code>	Alineación	left, center, right, justify
<code>line-height</code>	Espaciado entre líneas	1.2, 1.5, 2.0 (sin unidad)
<code>text-transform</code>	Transformación	uppercase, lowercase, capitalize

Manos a la obra

Mateo abrió el archivo `estilo.css` que Carolina había creado para el proyecto. Hasta ahora estaba vacío.

—Tu primera tarea —dijo Carolina—: dale estilo al encabezado principal y los párrafos de la portada.

Mateo escribió:

```

/* estilo.css – Cultura Viva */

body {
  font-family: Georgia, "Times New Roman", serif;
  color: #222;
  background-color: #fafafa;
}

h1 {
  font-size: 42px;
  color: #800000;
  text-align: center;
  text-transform: uppercase;
  letter-spacing: 2px;
}

h2 {
  font-size: 28px;
  color: #444;
  border-bottom: 2px solid #800000;
  padding-bottom: 5px;
}

p {
  font-size: 16px;
  line-height: 1.6;
  text-align: justify;
}

.destacado {
  background-color: #fff3e0;
  padding: 15px;
  font-style: italic;
  border-left: 4px solid #800000;
}

```

Mateo guardó el archivo y recargó el navegador. El sitio de "Cultura Viva" se veía diferente. No radicalmente, pero se notaba. Los títulos tenían presencia. Los párrafos se leían con fluidez. El fondo no era blanco puro, sino un crema suave.

—Se ve mejor —dijo, sonriendo.

—Se ve *humano* —corrigió Carolina—. Eso es lo que hace CSS: humaniza la tecnología.

Enigmas del Capítulo 1

Enigma 1.1: Escribe el CSS necesario para que todos los enlaces (`<a>`) del sitio sean de color verde oscuro (`#006400`), sin subrayado, y en negrita.

Enigma 1.2: Crea una clase llamada `.autor` que ponga el texto en cursiva, color gris (`#666`), tamaño 14px, y alineado a la derecha.

Enigma 1.3: ¿Cuál es la diferencia entre un selector de clase y un selector de ID? ¿Cuándo

usarías cada uno?

Enigma 1.4: Dado el siguiente HTML, escribe el CSS para que:

- El título principal sea azul marino, centrado, de 40px
- El subtítulo sea gris, tamaño 20px, en mayúsculas
- El párrafo con clase `intro` tenga fondo color `#eef` y un padding de 20px

```
<h1 id="titulo">Bienvenidos a Cultura Viva</h1>
<h2 class="subtitulo">Arte y pensamiento desde 1998</h2>
<p class="intro">La revista que conecta a los amantes del arte.</p>
<p>Conoce nuestras próximas exposiciones.</p>
```

Enigma 1.5: Explica con tus palabras qué significa que CSS sea "en cascada".

Capítulo 2: La Paleta de Colores

Tema: Colores, fondos, tipografía

A la mañana siguiente, Carolina llamó a Mateo a su escritorio. Tenía abierto un documento con la identidad visual de "Cultura Viva".

—Cada revista tiene una personalidad —dijo, señalando la pantalla—. "Cultura Viva" es una revista de arte y pensamiento. Debe verse elegante, seria pero no aburrida, moderna pero clásica. Colores, tipografía y fondos son la base de esa personalidad.

Colores en CSS

—Hay cuatro formas principales de definir colores en CSS —explicó Carolina, abriendo una paleta digital—.

Nombres de colores

```
color: red;
color: darkblue;
background-color: lightgray;
color: white;
```

—CSS reconoce 140 nombres de colores estándar. Son fáciles de recordar, pero limitados.

Colores hexadecimales (hex)

```
color: #FF5733;
color: #333333;
color: #fff;
color: #800000;
```

—El formato **hexadecimal** usa seis dígitos (o tres en versión corta) que representan los valores de rojo, verde y azul (RRGGBB). Cada par va de 00 a FF.

—`#FF0000` es rojo puro. `#00FF00` es verde puro. `#0000FF` es azul puro. Combinándolos obtienes cualquier color.

RGB

```
color: rgb(255, 87, 51);
color: rgb(51, 51, 51);
background-color: rgb(240, 240, 240);
```

—**RGB** es lo mismo que hexadecimal pero en números decimales del 0 al 255.

HSL

```
color: hsl(12, 100%, 60%);
background-color: hsl(0, 0%, 95%);
```

— **HSL** significa Hue (tono, 0-360°), Saturation (saturación, 0-100%), Lightness (luminosidad, 0-100%). Es más intuitivo para los diseñadores porque separa el tono de la intensidad.

—Mira —dijo Carolina, mostrando una tabla—:

Formato	Ejemplo	¿Cuándo usarlo?
Nombres	`red`, `white`	Prototipos rápidos
Hex	`#FF5733`	Lo más común en producción
RGB	`rgb(255,87,51)`	Cuando necesitas transparencia
HSL	`hsl(12,100%,60%)`	Cuando trabajas con paletas

Transparencia con rgba y hsla

—Puedes agregar un cuarto valor para la **opacidad** (alfa), de 0 a 1:

```
color: rgba(255, 87, 51, 0.8);
background-color: hsla(200, 50%, 50%, 0.3);
```

—0 es completamente transparente. 1 es completamente opaco.

background-color y background-image

—Ahora, hablemos de fondos —dijo Carolina, abriendo el CSS de la revista—.

```
/* Color de fondo */
body {
  background-color: #f5f0eb;
}

.header {
  background-color: #1a1a2e;
}

/* Imagen de fondo */
.banner {
  background-image: url("imagenes/banner-cultura.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

Propiedades de fondo importantes:

Propiedad	¿Qué hace?
`background-color`	Color de fondo
`background-image`	Imagen de fondo (url(...))
`background-size`	Tamaño: cover, contain, auto
`background-position`	Posición: center, top, left

<code>`background-repeat`</code>	Repetición: no-repeat, repeat, repeat-x
<code>`background`</code>	Propiedad abreviada (shorthand)

```
/* Shorthand: todo en una línea */
body {
  background: #f5f0eb url("textura.png") no-repeat center top;
}
```

Tipografía: font-family en profundidad

—El tipo de letra define la voz del texto —dijo Carolina, abriendo Google Fonts en el navegador—.

```
/* Web safe: fuentes que todos los sistemas tienen */
body {
  font-family: Arial, Helvetica, sans-serif;
}

/* Serif para títulos con elegancia */
h1, h2 {
  font-family: Georgia, "Times New Roman", serif;
}
```

Buenas prácticas con font-family

—Siempre incluye una **fuentes de respaldo** (fallback) y termina con una familia genérica:

```
font-family: "Open Sans", Arial, Helvetica, sans-serif;
```

—Si la primera fuente no está disponible, se usa la siguiente, y así sucesivamente.

Usando Google Fonts

—Mateo, vamos a usar una fuente elegante para los títulos de la revista.

Carolina fue a Google Fonts, seleccionó **Playfair Display** para títulos y **Lato** para el cuerpo, y copió el código:

```
<!-- En el <head> del HTML -->
<link href="https://fonts.googleapis.com/css2?family=Lato:wght@300;400;700&family=Playfair+Display:wght@400;700" rel="stylesheet">

/* En el CSS */
h1, h2, h3 {
  font-family: "Playfair Display", Georgia, serif;
  font-weight: 700;
}

body {
  font-family: "Lato", Arial, sans-serif;
  font-weight: 400;
}
```

Propiedades tipográficas avanzadas

Carolina mostró más propiedades que transformarían el texto:

```
.articulo {
  font-weight: 700;      /* 100 (light) a 900 (black) */
  font-style: italic;    /* normal, italic, oblique */
  line-height: 1.8;      /* Espaciado entre líneas */
  text-transform: uppercase; /* mayúsculas */
  text-decoration: underline; /* subrayado */
  letter-spacing: 2px;    /* Espaciado entre letras */
  word-spacing: 4px;      /* Espaciado entre palabras */
}
```

Ejemplo completo

Mateo aplicó la identidad visual de "Cultura Viva":

```
/* === CSS de Cultura Viva === */
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400;700;900&family=Lato:wght@300;400;700;900');

:root {
  --color-principal: #8B0000;
  --color-secundario: #1a1a2e;
  --color-fondo: #f5f0eb;
  --color-texto: #2d2d2d;
  --color-claro: #e8e0d5;
}

body {
  font-family: "Lato", Arial, sans-serif;
  color: var(--color-texto);
  background-color: var(--color-fondo);
  line-height: 1.7;
}

h1, h2, h3 {
  font-family: "Playfair Display", Georgia, serif;
  color: var(--color-principal);
}

h1 {
  font-size: 48px;
  font-weight: 900;
  letter-spacing: -0.5px;
  text-transform: capitalize;
}

h2 {
  font-size: 32px;
  font-weight: 700;
  letter-spacing: 1px;
}

.cita {
  font-style: italic;
}
```

```

font-size: 20px;
line-height: 1.6;
color: #555;
border-left: 4px solid var(--color-principal);
padding-left: 20px;
}

.fecha {
text-transform: uppercase;
font-size: 12px;
letter-spacing: 3px;
color: #888;
font-weight: 700;
}

```

Mateo guardó y recargó. La diferencia era notable. El sitio ahora tenía *personalidad*. Los títulos en Playfair Display se veían majestuosos; el cuerpo en Lato era limpio y legible. Los colores cálidos del fondo daban una sensación acogedora.

—Ahora sí parece una revista cultural —dijo Mateo.

—Ahora sí tiene identidad —corrigió Carolina—. La paleta de colores y la tipografía son la voz visual de la marca. Cada fuente, cada color, comunica algo.

Enigmas del Capítulo 2

Enigma 2.1: Escribe el CSS para que el fondo del sitio use un degradado lineal de `#f5foeb`` a `#e8d5c4``. Pista: usa `background: linear-gradient(...)``.

Enigma 2.2: Crea una regla CSS que haga que todos los enlaces (`<a>``) cambien de color cuando el usuario pase el mouse por encima. Usa `:hover``.

Enigma 2.3: Usando Google Fonts, agrega la fuente **Merriweather** para los artículos y **Montserrat** para los menús de navegación. Escribe tanto el HTML de enlace como el CSS.

Enigma 2.4: Convierte estos colores hex a rgb:

- `#FFFFFF``
- `#000000``
- `#8B0000``
- `#2E8B57``

Enigma 2.5: Explica por qué es importante usar `background-size: cover`` en una imagen de fondo. ¿Qué pasaría si no lo usas?

Capítulo 3: El Espacio entre Líneas

Tema: Modelo de caja (Box Model)

Pasó una semana. Mateo había mejorado el estilo del sitio significativamente. Los textos se veían bien, los colores funcionaban. Pero algo no andaba bien.

—Carolina —dijo Mateo, frustrado—. No entiendo por qué mis elementos no se alinean. Pongo un ``h2`` y un ``p`` uno al lado del otro y... mira esta página.

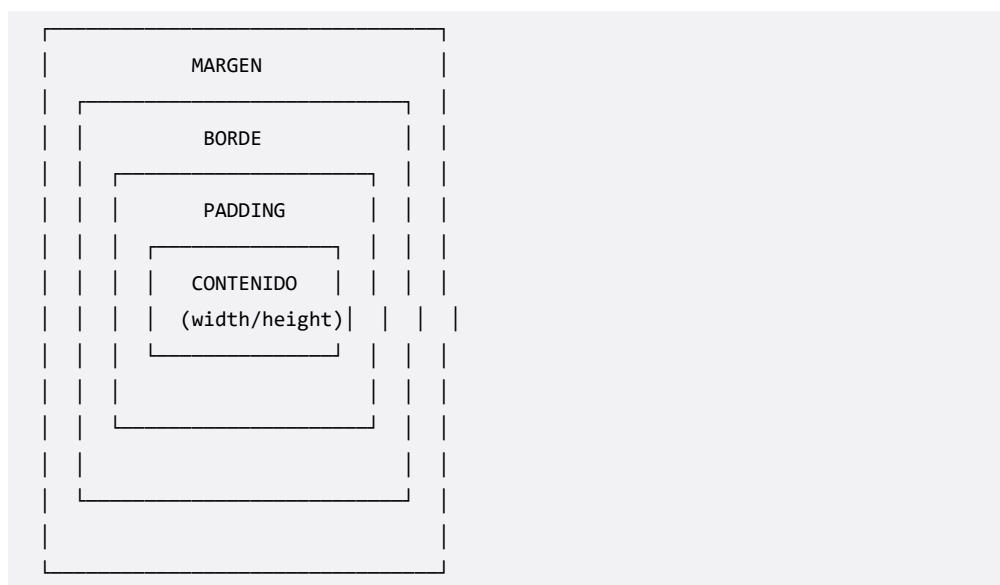
En la pantalla, la sección "Eventos" de "Cultura Viva" mostraba una lista de tarjetas anunciando exposiciones de arte. Pero en lugar de verse ordenadas, los elementos se superponían, los espacios eran irregulares, y el diseño se sentía "apretado" o "desordenado".

Carolina sonrió.

—Bienvenido al **modelo de caja**. Es el concepto más importante de CSS para entender layouts. Una vez que lo domines, todo lo demás tendrá sentido.

El Box Model

—Cada elemento HTML es una **caja** rectangular —dijo Carolina, dibujando en una pizarra virtual—. CSS ve el mundo como cajas dentro de cajas. Y cada caja tiene cuatro partes:



—De adentro hacia afuera:

1. **Contenido** — El contenido real del elemento (texto, imagen)
2. **Padding** — Espacio interno entre el contenido y el borde

3. **Border** — El borde que rodea el padding
4. **Margin** — Espacio externo entre el borde y otros elementos

Padding

—El **padding** es como el espacio dentro de una habitación entre los muebles y las paredes. Separa el contenido del borde.

```
.tarjeta {  
  padding-top: 10px;  
  padding-right: 20px;  
  padding-bottom: 10px;  
  padding-left: 20px;  
  
  /* Shorthand: arriba | derecha | abajo | izquierda */  
  padding: 10px 20px 10px 20px;  
  
  /* Shorthand: vertical | horizontal */  
  padding: 10px 20px;  
  
  /* Shorthand: todos igual */  
  padding: 15px;  
}
```

Margin

—El **margin** es el espacio *entre* elementos. Es como la distancia entre una casa y la siguiente.

```
.tarjeta {  
  margin-top: 20px;  
  margin-bottom: 30px;  
  
  /* Shorthand (misma lógica que padding) */  
  margin: 20px auto; /* auto centra horizontalmente */  
  margin: 10px 20px 30px 40px; /* arriba, derecha, abajo, izquierda */  
}
```

—Un detalle importante: el **margin collapse**. Cuando dos márgenes verticales se encuentran, se fusionan en uno solo (el mayor de los dos).

```
/* Si un h1 tiene margin-bottom: 30px */  
/* y un p tiene margin-top: 20px */  
/* El espacio entre ellos será 30px, no 50px */
```

Border

—El **border** es el borde visible de la caja.

```
.tarjeta {  
  border-width: 2px;  
  border-style: solid;  
  border-color: #8B0000;  
  border-radius: 8px;
```

```

/* Shorthand */
border: 2px solid #8B0000;

/* Borde solo en un lado */
border-bottom: 3px solid #1a1a2e;

/* Esquinas redondeadas */
border-radius: 10px;
border-radius: 50%; /* círculo perfecto si es un cuadrado */
}

```

Width y Height

—Puedes controlar el tamaño de la caja con `width` y `height`:

```

.tarjeta {
  width: 300px;
  height: 400px;
  max-width: 100%;
  min-height: 200px;
}

```

—Pero cuidado —advirtió Carolina—: el `width` que declaras es solo el ancho del **contenido**. El ancho total del elemento es contenido + padding + border + margin.

box-sizing: la salvación

—Aquí viene el problema que estabas teniendo —dijo Carolina—. Por defecto, CSS calcula el tamaño total como:

```

ancho total = width + padding-left + padding-right + border-left + border-right

```

—Eso significa que si declaras `width: 300px` y `padding: 20px`, el ancho real será 340px. Eso rompe los layouts.

—La solución: **`box-sizing: border-box`**:

```

/* Solución global */
* {
  box-sizing: border-box;
}

```

—Con `border-box`, el `width` que declaras incluye el padding y el borde. El ancho real será exactamente el que declaraste. El contenido se encoge para acomodar el padding.

Display: block, inline, inline-block

—El comportamiento de las cajas depende de su **display**:

```

/* Ocupa todo el ancho disponible. Respeta width y height */
display: block; /* <div>, <p>, <h1>, <section> */

```

```

/* Ocupa solo el espacio necesario. Ignora width y height */
display: inline; /* <span>, <a>, <strong> */

/* Como inline, pero respeta width, height, margin y padding */
display: inline-block; /* Lo mejor de ambos mundos */

```

box-shadow

—Las sombras dan profundidad visual:

```

.tarjeta {
  box-shadow: 10px 10px 20px rgba(0, 0, 0, 0.15);
  /* offset-x offset-y blur color */
}

/* Sombra solo en un lado */
.tarjeta {
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

/* Múltiples sombras */
.tarjeta {
  box-shadow: 0 2px 4px rgba(0,0,0,0.1), 0 4px 8px rgba(0,0,0,0.05);
}

```

Poniendo en práctica

Mateo aplicó el modelo de caja a las tarjetas de eventos de "Cultura Viva":

```

* {
  box-sizing: border-box;
}

.contenedor-eventos {
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
}

.tarjeta-evento {
  width: 350px;
  background-color: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 25px;
  margin: 15px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  display: inline-block;
  vertical-align: top;
}

.tarjeta-evento h3 {

```

```

font-family: "Playfair Display", serif;
color: #8B0000;
margin-top: 0;
border-bottom: 2px solid #eee;
padding-bottom: 10px;
}

.tarjeta-evento p {
  line-height: 1.5;
  margin-bottom: 0;
}

.tarjeta-evento .fecha {
  display: inline-block;
  background-color: #8B0000;
  color: white;
  padding: 5px 15px;
  border-radius: 20px;
  font-size: 12px;
  font-weight: 700;
  text-transform: uppercase;
}

```

Mateo guardó y recargó. Las tarjetas ahora se veían ordenadas, con espacio uniforme entre ellas. Cada tarjeta era una caja limpia con bordes redondeados y sombra suave.

—Entendí —dijo Mateo—. Cada elemento es una caja. Padding es el espacio interno. Margin es el espacio externo. Y `box-sizing: border-box`` hace que todo sea predecible.

—Exactamente —respondió Carolina—. El modelo de caja es la base de todo layout en CSS. Domínalo y dominarás el diseño web.

Enigmas del Capítulo 3

Enigma 3.1: Si tienes un ``div`` con ``width: 200px``, ``padding: 15px``, ``border: 3px solid black``, y ``box-sizing: content-box`` (valor por defecto), ¿cuál es el ancho total del elemento? ¿Y si usaras ``box-sizing: border-box``?

Enigma 3.2: Crea una tarjeta de perfil con:

- Ancho 280px
- Padding interno de 20px
- Borde redondeado de 12px
- Sombra suave
- Margen inferior de 25px
- Color de fondo blanco

Enigma 3.3: ¿Qué son los márgenes colapsados? Da un ejemplo de cuándo ocurren.

Enigma 3.4: Escribe el CSS para un botón con:

- Color de fondo ``#8B0000``

- Texto blanco, sin subrayado
- Padding: 12px arriba/abajo, 24px izquierda/derecha
- Borde: sin borde (`none`)
- Border-radius: 6px
- Al pasar el mouse, el fondo se aclara un poco

Enigma 3.5: Explica la diferencia entre `display: block`, `display: inline`, y `display: inline-block` con ejemplos de cuándo usar cada uno.

Capítulo 4: Poniendo Orden

Tema: Display y positioning

El sitio de "Cultura Viva" comenzaba a tomar forma. Los colores funcionaban, la tipografía era elegante, las tarjetas se alineaban correctamente. Pero Mateo enfrentaba un nuevo desafío: el diseño necesitaba un **header fijo** que permaneciera visible mientras el usuario hacía scroll, una **barra lateral** con los artículos más populares, y un layout general que no se desarmara al cambiar el tamaño de la ventana.

—Lo que necesitas —dijo Carolina, viendo a Mateo forcejear con el CSS— es entender **position** y controlar el flujo de los elementos.

Position: La posición de los elementos

—Por defecto, los elementos tienen `position: static`. Siguen el flujo normal del documento: uno tras otro, de arriba a abajo.

```
position: static; /* Valor por defecto: flujo normal */
```

—Pero a veces necesitas sacar elementos del flujo normal y posicionarlos en lugares específicos.

position: relative

```
.header {  
  position: relative;  
  top: 10px;    /* Se desplaza 10px hacia abajo desde su posición original */  
  left: 20px;   /* Se desplaza 20px hacia la derecha */  
}
```

—**Relative** mantiene el elemento en el flujo normal (su espacio original se reserva), pero puedes desplazarlo visualmente usando `top`, `right`, `bottom`, `left`.

position: absolute

```
.boton-flotante {  
  position: absolute;  
  top: 20px;  
  right: 20px;  
}
```

—**Absolute** saca el elemento del flujo normal. Se posiciona en relación al **ancestro**

posicionado más cercano (el primer ancestro con `position` diferente de `static`). Si no hay ninguno, se posiciona en relación al ``.

—Esto es muy útil para superponer elementos —dijo Carolina—. Por ejemplo, una etiqueta sobre una imagen:

```
.contenedor-imagen {  
  position: relative; /* El ancestro de referencia */  
}  
  
.etiqueta {  
  position: absolute;  
  top: 10px;  
  left: 10px;  
  background-color: rgba(139, 0, 0, 0.8);  
  color: white;  
  padding: 5px 10px;  
}
```

position: fixed

```
.header-principal {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 60px;  
  background-color: #1a1a2e;  
  color: white;  
  z-index: 1000;  
}
```

—**Fixed** es como absolute, pero se posiciona en relación a la ventana del navegador (viewport). El elemento permanece fijo incluso cuando haces scroll.

—Perfecto para el header —dijo Mateo.

position: sticky

```
.barra-lateral {  
  position: sticky;  
  top: 80px; /* Se vuelve fijo cuando el scroll llega a 80px del tope */  
}
```

—**Sticky** es un híbrido. El elemento se comporta como `relative` hasta que el scroll alcanza cierta posición, luego se comporta como `fixed`.

z-index: La profundidad

—Cuando los elementos se superponen, necesitas controlar qué elemento está adelante y cuál atrás.

```
.header-principal {  
  z-index: 1000; /* Mayor valor = más al frente */  
}
```

```

}

.modal {
  z-index: 2000;
}

.menu-desplegable {
  z-index: 1500;
}

```

—El **z-index** solo funciona en elementos posicionados (con `position` diferente de `static`).

Overflow: ¿Qué hacer con el contenido que se desborda?

```

.contenedor {
  width: 300px;
  height: 200px;
  overflow: hidden;      /* Oculta el contenido que desborda */
  overflow: scroll;      /* Muestra barras de scroll siempre */
  overflow: auto;       /* Muestra scroll solo cuando es necesario */
  overflow-x: hidden;   /* Control horizontal */
  overflow-y: auto;     /* Control vertical */
}

```

Float y Clear

—Antes de Flexbox y Grid, **float** era la forma principal de crear layouts. Hoy se usa menos, pero aún es útil para imágenes envueltas en texto.

```

.imagen-articulo {
  float: left; /* La imagen flota a la izquierda, el texto la rodea */
  margin-right: 20px;
  margin-bottom: 10px;
}

.clearfix::after {
  content: "";
  display: table;
  clear: both;
}

```

—**Clear** "limpia" los floats: evita que elementos posteriores se vean afectados por el float.

Construyendo el layout

Mateo aplicó todo lo aprendido para el layout de "Cultura Viva":

```

/* === Reset básico === */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

```

```

}

/* === Header fijo === */
.header-principal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 70px;
  background-color: #1a1a2e;
  color: white;
  z-index: 100;
  display: flex;
  align-items: center;
  padding: 0 30px;
}

.header-principal .logo {
  font-family: "Playfair Display", serif;
  font-size: 24px;
  color: white;
  text-decoration: none;
}

.header-principal nav {
  margin-left: auto;
}

.header-principal nav a {
  color: rgba(255, 255, 255, 0.8);
  text-decoration: none;
  margin-left: 20px;
  font-size: 14px;
  text-transform: uppercase;
  letter-spacing: 1px;
}

.header-principal nav a:hover {
  color: white;
}

/* === Espacio para el contenido (por el header fijo) === */
.contenido-principal {
  margin-top: 70px;
  padding: 30px;
  max-width: 1200px;
  margin-left: auto;
  margin-right: auto;
}

/* === Artículo con imagen flotante === */
.articulo {

```

```

    margin-bottom: 40px;
    overflow: hidden;
}

.articulo img {
    float: left;
    margin-right: 25px;
    margin-bottom: 15px;
    border-radius: 6px;
    max-width: 300px;
}

.articulo h2 {
    margin-bottom: 15px;
}

/* === Barra lateral sticky === */
.layout-columnas {
    display: flex;
    gap: 30px;
}

.columna-principal {
    flex: 2;
}

.barra-lateral {
    flex: 1;
    position: sticky;
    top: 90px;
    align-self: flex-start;
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.barra-lateral h3 {
    font-size: 18px;
    border-bottom: 2px solid #8B0000;
    padding-bottom: 8px;
    margin-bottom: 15px;
}

.barra-lateral ul {
    list-style: none;
}

.barra-lateral li {
    padding: 8px 0;
    border-bottom: 1px solid #eee;
}

```

```
.barra-lateral li a {  
  color: #333;  
  text-decoration: none;  
}  
  
.barra-lateral li a:hover {  
  color: #8B0000;  
}
```

Mateo probó el resultado. El header se mantenía fijo en la parte superior. La barra lateral seguía al usuario hasta cierto punto y luego se quedaba pegada. Las imágenes flotaban dentro de los artículos con el texto rodeándolas. El layout se veía profesional.

—Esto ya parece un sitio web de verdad —dijo Mateo.

—Esto ya es un sitio web de verdad —corrigió Carolina—. Ahora entiendes por qué la posición de los elementos es tan importante como su estilo. Controlar dónde está cada cosa es lo que separa un diseño amateur de uno profesional.

Enigmas del Capítulo 4

Enigma 4.1: Crea un botón "Volver arriba" que esté fijo en la esquina inferior derecha de la pantalla. Debe tener un z-index alto para que siempre esté visible.

Enigma 4.2: Explica la diferencia entre `position: absolute` y `position: fixed`. ¿Cuándo usarías cada uno?

Enigma 4.3: Diseña una tarjeta de artículo que tenga una etiqueta "Nuevo" superpuesta en la esquina superior izquierda usando `position: absolute`.

Enigma 4.4: ¿Qué problema resuelve `overflow: hidden` en un contenedor que tiene hijos con `float`?

Enigma 4.5: ¿Qué es el contexto de apilamiento? ¿Cómo afecta `z-index` a los elementos que están en diferentes contenedores posicionados?

Capítulo 5: Flexibilidad

Tema: Flexbox

—Mateo, ven aquí —dijo Carolina una mañana, señalando su pantalla—. La revista quiere una **galería de arte** en su sitio web. Necesito que las imágenes se muestren en filas, que se adapten al tamaño de la pantalla, y que estén bien espaciadas.

Mateo miró el diseño. Había imágenes de diferentes tamaños que debían organizarse en una cuadrícula flexible. Algunas filas tendrían cuatro imágenes, otras tres, según el espacio disponible.

—Podría usar `inline-block` otra vez —dijo Mateo, dubitativo.

—Podrías —respondió Carolina—, pero sería un dolor de cabeza. Hay una herramienta mucho mejor: **Flexbox**.

¿Qué es Flexbox?

—**Flexbox** (Flexible Box Layout) es un modelo de layout unidimensional —explicó Carolina—. Distribuye el espacio en una sola dirección: fila (horizontal) o columna (vertical). Es perfecto para alinear elementos, distribuir espacio, y crear diseños flexibles.

—Piénsalo así: tienes una caja flexible (el **flex container**) y dentro tienes elementos (los **flex items**). El contenedor decide cómo se distribuyen los hijos.

display: flex

```
.contenedor-galeria {  
  display: flex; /* Activa Flexbox en el contenedor */  
}
```

—Con solo esa línea, los hijos del contenedor se alinean horizontalmente uno al lado del otro.

flex-direction

```
/* Dirección principal: fila (por defecto) */  
flex-direction: row;  
  
/* Fila invertida */  
flex-direction: row-reverse;  
  
/* Columna */
```



```
flex-direction: column;

/* Columna invertida */
flex-direction: column-reverse;
```

—El **eje principal** (main axis) sigue la dirección de `flex-direction`. El **eje transversal** (cross axis) es perpendicular.

justify-content: Alineación en el eje principal

```
.contenedor {
  display: flex;
  justify-content: flex-start; /* Al inicio (por defecto) */
  justify-content: flex-end; /* Al final */
  justify-content: center; /* Centrado */
  justify-content: space-between; /* Espacio igual entre elementos */
  justify-content: space-around; /* Espacio alrededor de cada elemento */
  justify-content: space-evenly; /* Espacio uniforme */
}
```

Carolina mostró un diagrama textual:

```
flex-start:   [A][B][C]
center:       [A][B][C]
flex-end:     [A][B][C]
space-between: [A]   [B]   [C]
space-around: [A]   [B]   [C]
space-evenly: [A]  [B]  [C]
```

align-items: Alineación en el eje transversal

```
.contenedor {
  display: flex;
  align-items: stretch; /* Estirar al alto del contenedor (por defecto) */
  align-items: flex-start; /* Alinear al inicio del eje transversal */
  align-items: flex-end; /* Alinear al final */
  align-items: center; /* Centrar verticalmente */
  align-items: baseline; /* Alinear por la línea base del texto */
}
```

flex-wrap

```
.contenedor {
  display: flex;
  flex-wrap: nowrap; /* No saltan de línea (por defecto) */
  flex-wrap: wrap; /* Saltan a la siguiente línea si no caben */
  flex-wrap: wrap-reverse; /* Saltan en dirección inversa */
}
```

gap: Espacio entre elementos

—La forma más limpia de agregar espacio entre elementos flex:

```
.contenedor {
  display: flex;
  gap: 20px;           /* Espacio de 20px entre cada elemento */
  gap: 15px 30px;      /* row-gap y column-gap */
}
```

flex-grow, flex-shrink, flex-basis

—Estas propiedades se aplican a los **hijos** (flex items):

```
/* Los elementos crecen para llenar el espacio disponible */
.item {
  flex-grow: 1; /* Todos crecen por igual */
}

.item-doble {
  flex-grow: 2; /* Crece el doble que los demás */
}

/* Los elementos se encogen si no hay espacio */
.item {
  flex-shrink: 1; /* Valor por defecto */
}

/* Tamaño base antes de crecer o encoger */
.item {
  flex-basis: 200px; /* Tamaño inicial */
}

/* Shorthand */
.item {
  flex: 1;           /* flex-grow: 1, flex-shrink: 1, flex-basis: 0 */
  flex: 1 0 200px;   /* grow, shrink, basis */
}
```

order y align-self

```
/* Cambiar el orden visual (no el orden en HTML) */
.item-important {
  order: -1; /* Valores menores = aparecen primero */
}

.item-final {
  order: 1; /* Valores mayores = aparecen al final */
}

/* Alinear un elemento individual en el eje transversal */
```

```
.item-especial {
  align-self: flex-end; /* Solo este elemento se alinea al final */
}
```

La galería de arte

Mateo construyó la galería para "Cultura Viva":

```
/* === Contenedor de la galería === */
.galeria {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
  padding: 20px 0;
}

/* === Cada imagen en la galería === */
.galeria-item {
  flex: 1 0 250px; /* Crece, se encoge, base 250px */
  overflow: hidden;
  border-radius: 8px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
  transition: transform 0.3s ease;
}

.galeria-item:hover {
  transform: scale(1.03);
}

.galeria-item img {
  width: 100%;
  height: 200px;
  object-fit: cover; /* Recorta la imagen para llenar el espacio */
  display: block;
}

/* === Tarjetas de artículos con Flexbox === */
.fila-articulos {
  display: flex;
  gap: 25px;
  margin: 30px 0;
}

.tarjeta-articulo {
  flex: 1;
  background: white;
  border-radius: 8px;
  overflow: hidden;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
}

.tarjeta-articulo .contenido {
```

```
padding: 20px;
}

.tarjeta-articulo h3 {
margin-bottom: 10px;
}

.tarjeta-articulo .meta {
display: flex;
justify-content: space-between;
align-items: center;
margin-top: 15px;
padding-top: 10px;
border-top: 1px solid #eee;
font-size: 13px;
color: #888;
}

/* === Barra de navegación con Flexbox === */
.nav-links {
display: flex;
align-items: center;
gap: 25px;
list-style: none;
}

.nav-links li:last-child {
margin-left: auto; /* Empuja el último elemento al extremo opuesto */
}
```

Mateo probó la galería. Redimensionó la ventana y las imágenes se reordenaron solas. Cuando la pantalla era ancha, aparecían cuatro por fila. Cuando se hacía estrecha, pasaban a tres, luego a dos, luego a una. Sin una sola línea de JavaScript.

—Es mágico —dijo Mateo.

—No es magia —respondió Carolina—. Es Flexbox. Una de las herramientas más poderosas de CSS. Con Flexbox puedes hacer layouts que antes requerían trucos complicados o JavaScript.

—¿Entonces Flexbox sirve para todo? —preguntó Mateo.

—No todo —dijo Carolina, con una sonrisa misteriosa—. Para layouts bidimensionales —filas y columnas al mismo tiempo— hay algo incluso mejor. Pero eso será mañana.

Enigmas del Capítulo 5

Enigma 5.1: Usando Flexbox, crea un contenedor que tenga 3 elementos centrados horizontal y verticalmente dentro de él. El contenedor debe medir 400px de alto.

Enigma 5.2: Crea una barra de navegación horizontal con Flexbox donde:

- El logo esté a la izquierda
- Los enlaces de navegación estén centrados
- Un botón "Suscribirse" esté a la derecha

Enigma 5.3: ¿Cuál es la diferencia entre `justify-content: center` y `align-items: center`? ¿En qué eje trabaja cada uno?

Enigma 5.4: Diseña tres tarjetas de igual ancho usando `flex: 1`. La tarjeta del medio debe tener `flex: 2` (el doble de ancho). Explica por qué funciona.

Enigma 5.5: ¿Qué hace `flex-wrap: wrap` y por qué es importante para el diseño responsivo?

Capítulo 6: La Grilla Perfecta

Tema: CSS Grid

—Bien —dijo Carolina al día siguiente, abriendo el diseño de la portada digital de "Cultura Viva"—. La portada de la revista. Aquí es donde Flexbox se queda corto.

Mateo miró el diseño. Era un layout complejo: un título grande que ocupaba dos columnas, una imagen destacada, tres artículos secundarios, una cita lateral, y una barra de "lo más leído". Todo en una cuadrícula de filas y columnas que se entrecruzaban.

—Esto necesita **CSS Grid** —dijo Carolina—. Flexbox es unidimensional (una sola fila o columna). Grid es bidimensional: puedes controlar filas y columnas al mismo tiempo.

¿Qué es CSS Grid?

—**CSS Grid** es un sistema de layout bidimensional. Piensa en una hoja de papel cuadriculado donde puedes ubicar elementos en celdas específicas, combinando celdas para formar áreas más grandes.

display: grid

```
.contenedor-portada {  
  display: grid;  
}
```

—Así de simple se activa Grid. Pero para que haga algo útil, necesitas definir las columnas y filas.

grid-template-columns y grid-template-rows

```
.contenedor {  
  display: grid;  
  grid-template-columns: 200px 200px 200px; /* 3 columnas de 200px */  
  grid-template-rows: 100px 200px; /* 2 filas de 100px y 200px */  
}
```

La unidad fr

—La unidad más poderosa de Grid: **fr** (fracción). Divide el espacio disponible en fracciones.

```
.contenedor {  
  display: grid;
```

```

    grid-template-columns: 1fr 2fr 1fr; /* 4 fracciones: la del medio es el doble */
}

/* Equivalente a: 25% 50% 25% */

```

repeat() y minmax()

```

/* 3 columnas iguales */
grid-template-columns: repeat(3, 1fr);

/* 4 columnas, mínimo 200px, máximo 1fr (se expanden) */
grid-template-columns: repeat(4, minmax(200px, 1fr));

/* Columnas automáticas */
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
/* Crea tantas columnas de 250px como quepan */

```

gap en Grid

```

.contenedor {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;           /* Espacio entre filas y columnas */
    row-gap: 15px;       /* Solo entre filas */
    column-gap: 25px;     /* Solo entre columnas */
}

```

Colocar elementos: grid-column y grid-row

—Los hijos del grid se colocan automáticamente en celdas consecutivas. Pero puedes controlar su posición:

```

.header-portada {
    grid-column: 1 / 3;      /* De la línea 1 a la línea 3 (ocupa 2 columnas) */
    grid-row: 1 / 2;        /* De la línea 1 a la línea 2 (1 fila) */
}

.articulo-principal {
    grid-column: 1 / 3;      /* Ocupa 2 columnas */
    grid-row: 2 / 4;        /* Ocupa 2 filas */
}

.barra-lateral {
    grid-column: 3 / 4;      /* Tercera columna */
    grid-row: 1 / 4;        /* Toda la altura */
}

```

—También puedes usar `span`:

```

grid-column: span 2; /* Ocupa 2 columnas */
grid-row: span 3;    /* Ocupa 3 filas */

```

grid-area

—Puedes nombrar áreas del grid para hacerlo más legible:

```
.contenedor {
  display: grid;
  grid-template-columns: 2fr 1fr;
  grid-template-rows: auto auto auto;
  grid-template-areas:
    "header    header"
    "principal sidebar"
    "footer    footer";
  gap: 20px;
}

.header { grid-area: header; }
.main   { grid-area: principal; }
.aside  { grid-area: sidebar; }
.footer { grid-area: footer; }
```

—Esto hace que el CSS sea mucho más legible. Ves el layout directamente en el código.

place-items: Centrado en Grid

```
.contenedor {
  display: grid;
  place-items: center; /* Centra los hijos horizontal y verticalmente */
  height: 500px;
}
```

—`place-items` es shorthand para `align-items` y `justify-items`.

La portada digital

Mateo construyó el layout de la portada:

```
/* === Layout de la portada === */
.portada {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;
  grid-template-rows: auto auto auto;
  gap: 20px;
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
}

/* === Áreas de la portada === */
.titulo-principal {
  grid-column: 1 / 2;
  grid-row: 1 / 3;
  background: linear-gradient(135deg, #1a1a2e, #16213e);
  color: white;
```



```

padding: 40px;
border-radius: 12px;
display: flex;
flex-direction: column;
justify-content: center;
}

.titulo-principal h1 {
font-size: 48px;
line-height: 1.2;
margin-bottom: 20px;
}

.imagen-destacada {
grid-column: 2 / 4;
grid-row: 1 / 2;
}

.imagen-destacada img {
width: 100%;
height: 100%;
object-fit: cover;
border-radius: 12px;
}

/* === Los 3 artículos secundarios en fila === */
.articulos-secundarios {
grid-column: 1 / 4;
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 20px;
}

.articulo-secundario {
background: white;
padding: 25px;
border-radius: 8px;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

/* === Cita lateral === */
.cita-lateral {
grid-column: 1 / 2;
background-color: #f5f0eb;
padding: 30px;
border-radius: 8px;
border-left: 5px solid #8B0000;
font-style: italic;
font-size: 18px;
}

/* === Lo más leído === */

```

```

.mas-leido {
  grid-column: 2 / 4;
  background: white;
  padding: 25px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.mas-leido ol {
  list-style-position: inside;
}

.mas-leido li {
  padding: 10px 0;
  border-bottom: 1px solid #eee;
  font-weight: 600;
}

/* === Pie de página con Grid === */
.footer-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 30px;
  background-color: #1a1a2e;
  color: white;
  padding: 50px;
  margin-top: 50px;
  border-radius: 12px 12px 0 0;
}

.footer-grid h4 {
  font-family: "Playfair Display", serif;
  margin-bottom: 15px;
  color: #e8d5c4;
}

.footer-grid a {
  color: rgba(255, 255, 255, 0.7);
  text-decoration: none;
  display: block;
  margin-bottom: 8px;
}

```

Mateo probó el layout. Redimensionó la ventana y, aunque el grid se mantenía firme, notó que en pantallas pequeñas el diseño se rompía.

—Ya veo el problema —dijo—. Esto no funciona bien en móvil.

—Exacto —respondió Carolina—. Y de eso hablaremos mañana.

Enigmas del Capítulo 6

Enigma 6.1: Crea un grid de 4 columnas iguales usando `repeat()`. Cada columna debe tener

un gap de 15px.

Enigma 6.2: Usando ``grid-template-areas``, crea un layout con:

- Header en la parte superior (ocupa todo el ancho)
- Menú lateral a la izquierda (200px)
- Contenido principal al centro (ocupa el resto)
- Widget a la derecha (250px)
- Footer al fondo (todo el ancho)

Enigma 6.3: ¿Cuál es la diferencia entre ``grid-template-columns: repeat(auto-fit, minmax(200px, 1fr))`` y ``grid-template-columns: repeat(3, 1fr)``?

Enigma 6.4: Diseña un grid para una galería de imágenes donde:

- La primera imagen ocupe 2 columnas y 2 filas
- Las demás imágenes ocupen 1 columna y 1 fila cada una
- Usa ``grid-column`` y ``grid-row``

Enigma 6.5: Explica cuándo usarías Flexbox y cuándo usarías Grid. Da ejemplos concretos de cada uno.

Capítulo 7: Adaptándose al Mundo

Tema: Responsive Design y Media Queries

—Mateo —dijo Carolina una mañana, con el teléfono en la mano—, tenemos un problema. Mateo levantó la vista de su pantalla.

—Acabo de recibir un correo del editor de "Cultura Viva". Dice que varios lectores se han quejado. El sitio se ve bien en computadora, pero en el móvil... —Carolina giró su teléfono para mostrarle—. Mira.

En la pantalla del teléfono, el sitio de "Cultura Viva" era un desastre. El texto era minúsculo, las imágenes se desbordaban, los menús se superponían. Era ilegible.

—El problema —dijo Carolina— es que diseñaste para una pantalla específica. Pero la web se ve en miles de dispositivos diferentes: teléfonos, tablets, laptops, monitores enormes. Necesitas hacer que el sitio sea **responsivo**.

El viewport meta

—Lo primero: el **viewport**. Sin esta etiqueta, los navegadores móviles muestran el sitio como si fuera una pantalla de escritorio, obligando al usuario a hacer zoom.

```
<!-- En el <head> del HTML -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

—`width=device-width` hace que el ancho del viewport sea el del dispositivo. `initial-scale=1.0` establece el zoom inicial al 100%.

Media Queries

—Las **media queries** son la herramienta principal del diseño responsivo. Te permiten aplicar estilos diferentes según ciertas condiciones: ancho de pantalla, orientación, tipo de dispositivo.

```
/* Si la pantalla mide 768px o menos */
@media (max-width: 768px) {
  body {
    font-size: 14px;
  }

  .sidebar {
    display: none;
  }
}
```

```
}
```

—Sintaxis básica:

```
@media (condición) {  
    /* Reglas CSS que se aplican solo cuando se cumple la condición */  
}
```

Operadores comunes

```
/* Pantallas de hasta 600px */  
@media (max-width: 600px) { }  
  
/* Pantallas de 601px en adelante */  
@media (min-width: 601px) { }  
  
/* Entre 601px y 1024px */  
@media (min-width: 601px) and (max-width: 1024px) { }  
  
/* Orientación horizontal */  
@media (orientation: landscape) { }  
  
/* Pantallas táctiles */  
@media (hover: none) and (pointer: coarse) { }
```

Mobile-first vs Desktop-first

—Hay dos enfoques para escribir CSS responsivo —explicó Carolina—.

Desktop-first

```
/* Estilos base para desktop */  
.header { ... }  
  
/* Luego se sobreescriben para pantallas más pequeñas */  
@media (max-width: 768px) {  
    .header { ... }  
}  
  
@media (max-width: 480px) {  
    .header { ... }  
}
```

Mobile-first

```
/* Estilos base para móvil */  
.header { ... }  
  
/* Luego se mejoran para pantallas más grandes */  
@media (min-width: 768px) {  
    .header { ... }  
}
```

```
@media (min-width: 1024px) {
  .header { ... }
}
```

—Hoy en día se recomienda **mobile-first** —dijo Carolina—. Empiezas con el diseño más simple (móvil) y vas agregando complejidad a medida que hay más espacio.

Breakpoints comunes

—Los **breakpoints** son los puntos donde el diseño cambia. No hay valores mágicos, pero estos son puntos de referencia:

```
/* Móvil pequeño: < 480px */
/* Móvil: 480px - 768px */
/* Tablet: 768px - 1024px */
/* Desktop: 1024px - 1440px */
/* Pantallas grandes: > 1440px */
```

Imágenes responsivas

```
/* La imagen nunca supera el ancho de su contenedor */
img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

—Con `max-width: 100%`, las imágenes se encogen cuando el contenedor es más pequeño que la imagen, pero no se estiran más allá de su tamaño original.

Unidades relativas

—Usar píxeles fijos para todo hace que el diseño sea rígido. Las **unidades relativas** son clave para la flexibilidad:

Unidad	Relativa a	Ejemplo
<code>%</code>	El contenedor padre	<code>width: 50%</code>
<code>em</code>	El font-size del elemento	<code>padding: 2em</code>
<code>rem</code>	El font-size del <code><html></code>	<code>font-size: 1.2rem</code>
<code>vw</code>	1% del ancho del viewport	<code>width: 80vw</code>
<code>vh</code>	1% del alto del viewport	<code>height: 100vh</code>

—`rem` es excelente para tipografía porque escala con la configuración del usuario:

```
html {
  font-size: 16px; /* Tamaño base */
}

h1 {
  font-size: 2.5rem; /* 40px */
}
```

```

p {
  font-size: 1rem; /* 16px */
}

@media (max-width: 768px) {
  html {
    font-size: 14px; /* Reduce todo proporcionalmente */
  }
}

```

Haciendo "Cultura Viva" responsivo

Mateo reescribió el CSS para que fuera mobile-first:

```

/* === ESTILOS BASE (móvil primero) === */

html {
  font-size: 16px;
}

body {
  font-size: 1rem;
  margin: 0;
  padding: 15px;
}

/* Header: simple en móvil */
.header-principal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 60px;
  padding: 0 15px;
  display: flex;
  align-items: center;
}

.header-principal nav a {
  font-size: 12px;
  margin-left: 10px;
}

.contenido-principal {
  margin-top: 60px;
  padding: 15px;
}

/* Galería: una columna en móvil */
.galeria {
  display: grid;

```

```

    grid-template-columns: 1fr;
    gap: 15px;
}

/* Columnas: apiladas en móvil */
.layout-columns {
    display: block;
}

.barra-lateral {
    position: static; /* Vuelve al flujo normal */
    margin-top: 30px;
}

/* Footer: simple en móvil */
.footer-grid {
    grid-template-columns: 1fr;
    text-align: center;
}

/* === TABLET (>= 768px) === */
@media (min-width: 768px) {
    body {
        padding: 20px;
    }

    .galeria {
        grid-template-columns: repeat(2, 1fr);
    }

    .layout-columns {
        display: flex;
        gap: 25px;
    }

    .barra-lateral {
        position: sticky;
        top: 80px;
        flex: 0 0 300px;
    }

    .footer-grid {
        grid-template-columns: repeat(2, 1fr);
        text-align: left;
    }
}

/* === DESKTOP (>= 1024px) === */
@media (min-width: 1024px) {
    .contenido-principal {
        max-width: 1200px;
        margin: 70px auto 0;
    }
}

```



```

padding: 30px;
}

.galeria {
  grid-template-columns: repeat(3, 1fr);
}

.footer-grid {
  grid-template-columns: repeat(4, 1fr);
}

.header-principal {
  padding: 0 40px;
}

.header-principal nav a {
  font-size: 14px;
  margin-left: 25px;
}
}

/* === PANTALLAS GRANDES (>= 1440px) === */
@media (min-width: 1440px) {
  .galeria {
    grid-template-columns: repeat(4, 1fr);
  }

  html {
    font-size: 18px; /* Texto más grande en monitores grandes */
  }
}

```

Mateo guardó los cambios, agarró su teléfono, y abrió el sitio. El diseño se veía limpio. El menú era legible. Las imágenes se ajustaban al ancho de la pantalla. Hizo scroll, giró el teléfono a horizontal, y todo seguía funcionando.

—Ahora sí —dijo Carolina, asintiendo con satisfacción—. Esto es un sitio web moderno.

Enigmas del Capítulo 7

Enigma 7.1: Usando media queries, haz que un menú de navegación horizontal se convierta en un menú vertical apilado cuando la pantalla sea menor a 600px.

Enigma 7.2: ¿Cuál es la diferencia entre `em` y `rem`? ¿Cuándo usarías cada uno?

Enigma 7.3: Diseña una tarjeta que en desktop tenga `width: 300px` y esté en `inline-block`, pero en móvil (< 480px) ocupe el 100% del ancho.

Enigma 7.4: Explica por qué es importante la etiqueta `

Enigma 7.5: Crea un hero (banner principal) que ocupe `100vh` de alto (toda la pantalla) y

tenga una imagen de fondo responsiva.

Capítulo 8: El Toque Mágico

Tema: Transiciones y animaciones

El sitio de "Cultura Viva" funcionaba. Se veía bien en todos los dispositivos. Pero había algo que Mateo notaba que faltaba: movimiento.

—Se siente estático —dijo Mateo, moviendo el mouse por la pantalla—. Los botones están ahí, pero no reaccionan. Los elementos aparecen, pero no hay fluidez.

—Eso es porque no tienes **transiciones** ni **animaciones** —dijo Carolina—. El movimiento en la web no es decorativo: guía al usuario, le da feedback, hace la experiencia más natural.

Transition: La transición suave

—Las **transiciones** permiten que los cambios de estilo ocurran de forma gradual en lugar de instantánea.

```
.boton {
  background-color: #880000;
  color: white;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
  transition: all 0.3s ease;
}

.boton:hover {
  background-color: #a52a2a;
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(139, 0, 0, 0.3);
}
```

—Ahora, al pasar el mouse, el botón cambia de color suavemente y se eleva un poco.

Sintaxis de transition

```
.elemento {
  /* Propiedad específica */
  transition: background-color 0.3s ease;

  /* Múltiples propiedades */
  transition: background-color 0.3s ease, transform 0.2s ease-in;
```

```

/* Todas las propiedades */
transition: all 0.3s ease;

/* Shorthand completo: propiedad duración función retardo */
transition: background-color 0.3s ease 0.1s;
}

```

Funciones de temporización (easing)

Función	Comportamiento
`ease`	Inicio suave, fin suave (por defecto)
`linear`	Velocidad constante
`ease-in`	Inicio lento, aceleración al final
`ease-out`	Inicio rápido, desaceleración al final
`ease-in-out`	Lento al inicio y al final
`cubic-bezier(n,n,n,n)`	Curva personalizada

Transform: Transformar elementos

—**Transform** modifica la apariencia de un elemento sin afectar el flujo del documento:

```

/* Rotar */
.icono:hover {
  transform: rotate(45deg);
}

/* Escalar (agrandar o encoger) */
.tarjeta:hover {
  transform: scale(1.05);
}

/* Mover */
.boton:active {
  transform: translateY(2px); /* Efecto de presionar */
}

/* Inclinar */
.caja {
  transform: skew(10deg, 5deg);
}

/* Múltiples transformaciones */
.elemento:hover {
  transform: rotate(5deg) scale(1.1) translateX(10px);
}

```

Animation y @keyframes

—Para movimientos más complejos o que se repiten automáticamente, usas **animaciones**:

```

/* 1. Definir la animación con @keyframes */

```

```

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* 2. Aplicarla a un elemento */
.articulo {
  animation: fadeIn 0.8s ease-out;
}

```

Keyframes con porcentajes

—Puedes definir múltiples puntos intermedios:

```

@keyframes pulse {
  0% {
    transform: scale(1);
    opacity: 1;
  }
  50% {
    transform: scale(1.1);
    opacity: 0.8;
  }
  100% {
    transform: scale(1);
    opacity: 1;
  }
}

.boton-suscribirse {
  animation: pulse 2s ease-in-out infinite;
}

```

Propiedades de animación

```

.elemento {
  animation-name: fadeIn;           /* Nombre del @keyframes */
  animation-duration: 0.8s;         /* Duración */
  animation-timing-function: ease-out; /* Curva de aceleración */
  animation-delay: 0.2s;            /* Retardo antes de empezar */
  animation-iteration-count: 1;     /* Número de repeticiones: 1, 2, infinite */
  animation-direction: normal;      /* normal, reverse, alternate */
  animation-fill-mode: forwards;    /* Mantiene el estado final */
  animation-play-state: running;     /* running, paused */

  /* Shorthand */
  animation: fadeIn 0.8s ease-out 0.2s 1 normal forwards;
}

```

```
}
```

Animaciones que se activan con scroll

—Aunque eso normalmente requiere JavaScript, puedes hacer animaciones simples que se activen con hover:

```
@keyframes slideInLeft {  
  from {  
    opacity: 0;  
    transform: translateX(-50px);  
  }  
  to {  
    opacity: 1;  
    transform: translateX(0);  
  }  
}  
  
.tarjeta {  
  animation: slideInLeft 0.5s ease-out;  
  animation-fill-mode: both;  
}  
  
.tarjeta:nth-child(2) {  
  animation-delay: 0.2s;  
}  
  
.tarjeta:nth-child(3) {  
  animation-delay: 0.4s;  
}
```

Dale vida a "Cultura Viva"

Mateo agregó movimiento a todo el sitio:

```
/* === Animaciones globales === */  
@keyframes fadeInUp {  
  from {  
    opacity: 0;  
    transform: translateY(30px);  
  }  
  to {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}  
  
@keyframes fadeInLeft {  
  from {  
    opacity: 0;  
    transform: translateX(-30px);  
  }  
  to {
```

```

        opacity: 1;
        transform: translateX(0);
    }
}

@keyframes pulse {
    0%, 100% { transform: scale(1); }
    50% { transform: scale(1.05); }
}

/* === Animaciones en elementos específicos === */
.hero-titulo {
    animation: fadeInUp 1s ease-out;
}

.hero-subtitulo {
    animation: fadeInUp 1s ease-out 0.3s both;
}

.hero-boton {
    animation: fadeInUp 1s ease-out 0.6s both;
}

/* === Transiciones en elementos interactivos === */
.boton {
    transition: all 0.3s ease;
    cursor: pointer;
}

.boton:hover {
    transform: translateY(-3px);
    box-shadow: 0 6px 20px rgba(0, 0, 0, 0.15);
}

.boton:active {
    transform: translateY(-1px);
}

/* === Tarjetas con efecto hover === */
.tarjeta-evento {
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.tarjeta-evento:hover {
    transform: translateY(-5px);
    box-shadow: 0 12px 30px rgba(0, 0, 0, 0.12);
}

/* === Galería: hover con zoom === */
.galeria-item {
    overflow: hidden;
    border-radius: 8px;

```

```

}

.galeria-item img {
  transition: transform 0.4s ease;
}

.galeria-item:hover img {
  transform: scale(1.1);
}

/* === Botón de suscripción con pulso === */
.boton-suscripcion {
  background-color: #8B0000;
  color: white;
  border: none;
  padding: 15px 30px;
  border-radius: 30px;
  font-size: 16px;
  font-weight: 700;
  cursor: pointer;
  transition: all 0.3s ease;
  animation: pulse 2s ease-in-out infinite;
}

.boton-suscripcion:hover {
  animation: none; /* Detiene el pulso al hacer hover */
  background-color: #a52a2a;
  transform: scale(1.05);
}

/* === Barra de progreso al hacer scroll === */
/* (Esto se vería con JavaScript, pero el CSS está listo) */
.barra-progreso {
  height: 3px;
  background: linear-gradient(90deg, #8B0000, #cc3333);
  position: fixed;
  top: 0;
  left: 0;
  z-index: 9999;
  transition: width 0.1s linear;
}

```

Mateo recargó el sitio. Los títulos aparecían con una animación suave. Los botones reaccionaban al mouse con elegancia. Las imágenes de la galería se agrandaban suavemente al pasar el cursor. Los artículos se deslizaban hacia arriba al cargar la página.

—Ahora el sitio respira —dijo Mateo.

—Exacto —respondió Carolina—. El movimiento hace que el sitio se sienta vivo. No es decoración: es **comunicación**. Un botón que se eleva dice "puedes hacerme clic". Una tarjeta que se agranda dice "soy importante". Las animaciones guían al usuario sin palabras.

Enigmas del Capítulo 8

Enigma 8.1: Crea una animación llamada ``bounce`` que haga que un elemento rebote (suba y baje) indefinidamente. Pista: usa ``translateY``.

Enigma 8.2: Diseña un botón con las siguientes transiciones:

- Color de fondo cambia suavemente (0.3s)
- Escala aumenta ligeramente (0.2s)
- Aparece una sombra (0.3s)

Todo debe ocurrir al hacer hover.

Enigma 8.3: Crea una animación de **carga** (loading spinner) usando solo CSS. Un círculo que gira 360 grados infinitamente.

Enigma 8.4: ¿Cuál es la diferencia entre ``transition`` y ``animation``? ¿Cuándo usarías cada uno?

Enigma 8.5: Explica qué hace ``animation-fill-mode: forwards`` y por qué es útil.

Capítulo 9: Detalles que Importan

Tema: Pseudo-clases y pseudo-elementos

—El sitio se ve bien —dijo Carolina, recorriendo las páginas—. Pero los grandes diseñadores se distinguen en los detalles.

—¿Qué detalles? —preguntó Mateo.

—Ven. Mira esta lista de artículos. ¿Notas algo?

Mateo observó. La lista era funcional, pero carecía de los pequeños toques que hacen que un diseño se sienta pulido: los bordes de las tablas no tenían ritmo, los enlaces no daban feedback visual, no había indicadores de posición ni elementos decorativos sutiles.

—Hora de aprender sobre **pseudo-clases** y **pseudo-elementos** —dijo Carolina.

Pseudo-clases

—Las **pseudo-clases** describen un estado especial de un elemento. Se escriben con `:` después del selector.

:hover — El estado más usado

```
.enlace:hover {
  color: #8B0000;
  text-decoration: underline;
}

.tarjeta:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 25px rgba(0, 0, 0, 0.15);
}
```

—Se activa cuando el usuario pasa el mouse sobre el elemento.

:focus y **:active**

```
/* Cuando el elemento recibe foco (teclado o clic) */
input:focus {
  border-color: #8B0000;
  outline: 2px solid rgba(139, 0, 0, 0.3);
  box-shadow: 0 0 4px rgba(139, 0, 0, 0.1);
}

/* Cuando el elemento está siendo clickeado */
```

```
.boton:active {
  transform: scale(0.98);
}
```

—`:focus` es importante para accesibilidad: los usuarios de teclado necesitan ver qué elemento está activo.

:nth-child() — Selectores posicionales

```
/* El tercer hijo de su padre */
li:nth-child(3) {
  font-weight: bold;
}

/* Los hijos pares */
li:nth-child(even) {
  background-color: #f9f9f9;
}

/* Los hijos impares */
li:nth-child(odd) {
  background-color: white;
}

/* Cada 3 elementos, empezando por el 2 */
li:nth-child(3n+2) {
  color: #8B0000;
}
```

Carolina mostró una tabla con ejemplos:

Selector	Selecciona
`:first-child`	El primer hijo
`:last-child`	El último hijo
`:nth-child(2)`	El segundo hijo
`:nth-child(odd)`	Hijos en posición impar
`:nth-child(even)`	Hijos en posición par
`:nth-child(3n)`	Cada 3 elementos
`:nth-last-child(2)`	El segundo desde el final

:not() — La negación

```
/* Todos los párrafos excepto los de clase .intro */
p:not(.intro) {
  color: #555;
}

/* Todos los elementos excepto el último */
li:not(:last-child) {
  border-bottom: 1px solid #eee;
}
```

Pseudo-elementos

—Los **pseudo-elementos** crean elementos "fantasma" que no existen en el HTML. Se escriben con `::` (doble dos puntos).

::before y ::after

—Los más poderosos. Insertan contenido antes o después de un elemento.

```
/* Agregar una comilla decorativa antes de las citas */
blockquote::before {
  content: "\201C"; /* Comilla doble izquierda Unicode */
  font-size: 60px;
  color: #8B0000;
  font-family: Georgia, serif;
  position: absolute;
  top: -10px;
  left: -10px;
}

/* Agregar un icono de enlace externo */
a[target="_blank"]::after {
  content: " ";
  font-size: 12px;
}

/* Decoración de listas personalizadas */
.lista-articulos li::before {
  content: "";
  color: #8B0000;
  margin-right: 10px;
}
```

—`::before` crea un elemento como primer hijo. `::after` lo crea como último hijo. Ambos requieren la propiedad `content` para existir.

::first-line y ::first-letter

```
/* Primera línea del párrafo */
p::first-line {
  font-weight: bold;
  color: #333;
}

/* Primera letra (capital) */
.articulo p::first-letter {
  font-size: 48px;
  font-weight: 900;
  color: #8B0000;
  float: left;
  margin-right: 8px;
  line-height: 1;
  font-family: "Playfair Display", serif;
}
```

—Esto crea un efecto de **capitular** (letra capital) como en los libros antiguos.

Detalles que marcan la diferencia en "Cultura Viva"

Mateo aplicó pseudo-clases y pseudo-elementos en todo el sitio:

```
/* === Tabla de eventos con filas alternadas === */
.tabla-eventos tr:nth-child(even) {
  background-color: #f9f5f0;
}

.tabla-eventos tr:hover {
  background-color: #f0e8dd;
}

.tabla-eventos td {
  padding: 12px 15px;
  border-bottom: 1px solid #e0d6cc;
}

.tabla-eventos tr:last-child td {
  border-bottom: none;
}

/* === Estilo de listas con indicador visual === */
.lista-categorias li {
  padding: 10px 0;
  border-bottom: 1px solid #eee;
}

.lista-categorias li:last-child {
  border-bottom: none;
}

.lista-categorias li::before {
  content: "►";
  color: #8B0000;
  margin-right: 10px;
  font-weight: bold;
}

/* === Enlaces con feedback visual === */
.enlace-articulo {
  color: #1a1a2e;
  text-decoration: none;
  transition: color 0.2s ease;
  position: relative;
}

.enlace-articulo:hover {
  color: #8B0000;
}
```

```

/* Línea decorativa que aparece al hacer hover */
.enlace-articulo::after {
  content: "";
  position: absolute;
  bottom: -2px;
  left: 0;
  width: 0;
  height: 2px;
  background-color: #8B0000;
  transition: width 0.3s ease;
}

.enlace-articulo:hover::after {
  width: 100%;
}

/* === Capital inicial en artículos destacados === */
.articulo-destacado p:first-of-type::first-letter {
  font-size: 3em;
  font-weight: 900;
  color: #8B0000;
  float: left;
  margin-right: 10px;
  margin-top: 5px;
  line-height: 0.8;
  font-family: "Playfair Display", serif;
}

/* === Tooltips con CSS === */
[data-tooltip] {
  position: relative;
  cursor: help;
}

[data-tooltip]:hover::after {
  content: attr(data-tooltip);
  position: absolute;
  bottom: 100%;
  left: 50%;
  transform: translateX(-50%);
  background-color: #333;
  color: white;
  padding: 5px 10px;
  border-radius: 4px;
  font-size: 12px;
  white-space: nowrap;
  z-index: 100;
}

/* === Formulario de suscripción === */
.campo-formulario {
  border: 2px solid #ddd;
}

```

```

border-radius: 6px;
padding: 12px 15px;
font-size: 16px;
transition: border-color 0.3s ease, box-shadow 0.3s ease;
}

.campo-formulario:focus {
border-color: #8B0000;
outline: none;
box-shadow: 0 0 0 3px rgba(139, 0, 0, 0.1);
}

.campo-formulario:invalid {
border-color: #e74c3c;
}

.campo-formulario:valid {
border-color: #27ae60;
}

/* === Mensajes de error con pseudo-clase === */
.error-mensaje {
display: none;
color: #e74c3c;
font-size: 13px;
margin-top: 5px;
}

.campo-formulario:invalid ~ .error-mensaje {
display: block;
}

```

—Fíjate —dijo Carolina— cómo usamos `::after` para crear la línea decorativa que crece bajo los enlaces. No hay un solo elemento extra en el HTML. Todo el detalle visual está en el CSS.

Mateo observó el resultado. Los pequeños detalles transformaban la experiencia: las filas de la tabla se leían con facilidad, los enlaces tenían una animación sutil, los formularios daban feedback visual, las listas tenían indicadores elegantes.

—Los detalles no son detalles —dijo Mateo, recordando una frase que había leído en algún lado—. Los detalles son el diseño.

—Exactamente —sonrió Carolina—. Ahora entiendes.

Enigmas del Capítulo 9

Enigma 9.1: Usando `::before`, agrega un icono de estrella () antes de cada elemento de una lista con clase `.destacados`.

Enigma 9.2: Crea un efecto de "línea que crece" (underline animation) para los enlaces del menú de navegación usando `::after` y `transition`.

Enigma 9.3: Usando `:nth-child`, haz que en una galería de 6 imágenes, la tercera y la sexta

tengan un borde de color diferente (por ejemplo, dorado).

Enigma 9.4: Diseña un efecto de "tooltip" (información emergente) usando `::after` y el atributo `data-tooltip`. El tooltip debe aparecer al hacer hover y desvanecerse suavemente.

Enigma 9.5: Explica la diferencia entre pseudo-clases (`:hover`) y pseudo-elementos (`::before`). Da dos ejemplos de cada uno.

Capítulo 10: El Lanzamiento

Tema: Proyecto final — Maquetar la portada completa

Era viernes, el último día antes del lanzamiento del sitio web de "Cultura Viva". Mateo llegó a PixelLab a las 7:30 de la mañana, dos horas antes de lo habitual. El café de la oficina aún no estaba listo, así que se preparó un té y se sentó frente a su estación de trabajo.

En la pantalla tenía el diseño final que Carolina le había pasado: la **portada completa** de la revista digital. Era un layout complejo con múltiples secciones: hero, artículos destacados, galería, eventos, suscripción, y footer. Todo debía funcionar en cualquier dispositivo.

—Hoy no hay lecciones —dijo Carolina, apareciendo detrás de él con su café—. Hoy pones a prueba todo lo que aprendiste.

Mateo asintió. Abrió el archivo `index.html` de la portada y, al lado, su archivo `estilo.css`. Había llegado el momento.

El proyecto final

El HTML de la portada tenía esta estructura:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cultura Viva – Arte y Pensamiento</title>
  <link rel="stylesheet" href="estilo.css">
</head>
<body>

  <header class="header-principal">
    <a href="#" class="logo">Cultura Viva</a>
    <nav>
      <a href="#">Inicio</a>
      <a href="#">Arte</a>
      <a href="#">Literatura</a>
      <a href="#">Música</a>
      <a href="#">Eventos</a>
    </nav>
    <a href="#" class="btn-suscripcion">Suscribirse</a>
  </header>
```

```

<main>
  <!-- Hero principal -->
  <section class="hero">
    <div class="hero-contenido">
      <span class="hero-etiqueta">Edición Julio 2026</span>
      <h1 class="hero-titulo">El Renacimiento Digital del Arte Latinoamericano</h1>
      <p class="hero-subtitulo">Exploramos cómo los artistas contemporáneos están transformando la identidad digital.</p>
      <a href="#" class="btn-primario">Leer artículo</a>
    </div>
    <div class="hero-imagen">
      
    </div>
  </section>

  <!-- Artículos destacados -->
  <section class="articulos-destacados">
    <h2 class="seccion-titulo">Artículos Destacados</h2>
    <div class="grid-articulos">
      <article class="tarjeta-articulo">
        
        <div class="tarjeta-contenido">
          <span class="tarjeta-categoria">Literatura</span>
          <h3>La Narrativa del Siglo XXI</h3>
          <p>Autores que están redefiniendo la literatura latinoamericana desde las plataformas digitales.</p>
          <a href="#" class="enlace-articulo">Leer más →</a>
        </div>
      </article>
      <article class="tarjeta-articulo">
        
        <div class="tarjeta-contenido">
          <span class="tarjeta-categoria">Música</span>
          <h3>Sonidos de Transformación</h3>
          <p>La fusión de ritmos tradicionales con producción digital está creando un nuevo paisaje sonoro.</p>
          <a href="#" class="enlace-articulo">Leer más →</a>
        </div>
      </article>
      <article class="tarjeta-articulo">
        
        <div class="tarjeta-contenido">
          <span class="tarjeta-categoria">Arte</span>
          <h3>Galerías sin Paredes</h3>
          <p>El auge de las exposiciones virtuales y su impacto en la democratización del arte.</p>
          <a href="#" class="enlace-articulo">Leer más →</a>
        </div>
      </article>
    </div>
  </section>

  <!-- Galería de arte -->
  <section class="galeria-seccion">
    <h2 class="seccion-titulo">Galería Destacada</h2>
    <div class="galeria">

```

```

    <div class="galeria-item"></div>
    <div class="galeria-item"></div>
    <div class="galeria-item"></div>
    <div class="galeria-item"></div>
    <div class="galeria-item"></div>
    <div class="galeria-item"></div>
  </div>
</section>

```

```

<!-- Eventos -->
<section class="eventos-seccion">
  <h2 class="seccion-titulo">Próximos Eventos</h2>
  <div class="lista-eventos">
    <div class="tarjeta-evento">
      <div class="evento-fecha">
        <span class="evento-dia">28</span>
        <span class="evento-mes">JUL</span>
      </div>
      <div class="evento-info">
        <h3>Exposición: Colores del Ande</h3>
        <p>Galería de Arte Contemporáneo, Miraflores</p>
      </div>
    </div>
    <div class="tarjeta-evento">
      <div class="evento-fecha">
        <span class="evento-dia">05</span>
        <span class="evento-mes">AGO</span>
      </div>
      <div class="evento-info">
        <h3>Conferencia: Literatura Digital</h3>
        <p>Centro Cultural PUCP, San Miguel</p>
      </div>
    </div>
    <div class="tarjeta-evento">
      <div class="evento-fecha">
        <span class="evento-dia">19</span>
        <span class="evento-mes">AGO</span>
      </div>
      <div class="evento-info">
        <h3>Concierto: Fusión Andina</h3>
        <p>Teatro Municipal, Lima Centro</p>
      </div>
    </div>
  </div>
</section>

```

```

<!-- Suscripción -->
<section class="suscripcion-seccion">
  <div class="suscripcion-contenido">
    <h2>No te pierdas ninguna edición</h2>
    <p>Suscríbete a Cultura Viva y recibe cada mes lo mejor del arte y el pensamiento contemporáneo.</p>
    <form class="suscripcion-form">

```

```

        <input type="email" class="campo-formulario" placeholder="Tu correo electrónico">
        <button type="submit" class="btn-suscripcion">Suscribirse</button>
    </form>
</div>
</section>
</main>

<footer class="footer-principal">
    <div class="footer-grid">
        <div>
            <h4>Cultura Viva</h4>
            <p>Arte y pensamiento desde 1998</p>
        </div>
        <div>
            <h4>Secciones</h4>
            <a href="#">Arte</a>
            <a href="#">Literatura</a>
            <a href="#">Música</a>
            <a href="#">Eventos</a>
        </div>
        <div>
            <h4>Síguenos</h4>
            <a href="#">Facebook</a>
            <a href="#">Instagram</a>
            <a href="#">Twitter</a>
        </div>
        <div>
            <h4>Contacto</h4>
            <a href="#">contacto@culturaviva.pe</a>
            <a href="#">+51 1 555 0123</a>
        </div>
    </div>
    <div class="footer-bottom">
        <p>&copy; 2026 Cultura Viva. Todos los derechos reservados.</p>
    </div>
</footer>

</body>
</html>

```

Mateo respiró hondo y comenzó a escribir el CSS completo:

```

/* === CSS COMPLETO – CULTURA VIVA PORTADA === */

/* === RESET === */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

/* === TIPOGRAFÍA === */
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400;700;900&family=Lato:wght@300;400;700;900');

```

```

:root {
  --color-primario: #8B0000;
  --color-secundario: #1a1a2e;
  --color-fondo: #f5f0eb;
  --color-texto: #2d2d2d;
  --color-claro: #e8e0d5;
  --fuente-titulos: "Playfair Display", Georgia, serif;
  --fuente-cuerpo: "Lato", Arial, sans-serif;
}

html {
  font-size: 16px;
  scroll-behavior: smooth;
}

body {
  font-family: var(--fuente-cuerpo);
  color: var(--color-texto);
  background-color: var(--color-fondo);
  line-height: 1.7;
}

img {
  max-width: 100%;
  height: auto;
  display: block;
}

a {
  color: inherit;
  text-decoration: none;
}

/* === HEADER === */
.header-principal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 70px;
  background-color: var(--color-secundario);
  color: white;
  display: flex;
  align-items: center;
  padding: 0 30px;
  z-index: 1000;
}

.logo {
  font-family: var(--fuente-titulos);
  font-size: 24px;

```

```

    font-weight: 700;
    color: white;
}

.header-principal nav {
    display: flex;
    gap: 25px;
    margin-left: 50px;
}

.header-principal nav a {
    font-size: 13px;
    text-transform: uppercase;
    letter-spacing: 1.5px;
    color: rgba(255, 255, 255, 0.7);
    transition: color 0.3s ease;
}

.header-principal nav a:hover {
    color: white;
}

.btn-suscripcion {
    margin-left: auto;
    background-color: var(--color-primario);
    color: white;
    padding: 10px 22px;
    border-radius: 25px;
    font-size: 13px;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 1px;
    transition: background-color 0.3s ease, transform 0.3s ease;
}

.btn-suscripcion:hover {
    background-color: #a52a2a;
    transform: translateY(-2px);
}

/* === HERO === */
.hero {
    margin-top: 70px;
    display: grid;
    grid-template-columns: 1fr 1fr;
    min-height: 500px;
    background-color: var(--color-secundario);
    color: white;
    overflow: hidden;
}

.hero-contenido {

```

```

padding: 60px 50px;
display: flex;
flex-direction: column;
justify-content: center;
animation: fadeInUp 1s ease-out;
}

.hero-etiqueta {
display: inline-block;
font-size: 12px;
text-transform: uppercase;
letter-spacing: 3px;
color: var(--color-primario);
font-weight: 700;
margin-bottom: 20px;
}

.hero-titulo {
font-family: var(--fuente-titulos);
font-size: 42px;
font-weight: 900;
line-height: 1.2;
margin-bottom: 20px;
}

.hero-subtitulo {
font-size: 18px;
line-height: 1.6;
color: rgba(255, 255, 255, 0.7);
margin-bottom: 30px;
}

.btn-primario {
display: inline-block;
background-color: var(--color-primario);
color: white;
padding: 14px 32px;
border-radius: 6px;
font-weight: 700;
transition: all 0.3s ease;
align-self: flex-start;
}

.btn-primario:hover {
background-color: #a52a2a;
transform: translateY(-2px);
box-shadow: 0 6px 20px rgba(139, 0, 0, 0.4);
}

.hero-imagen {
overflow: hidden;
}

```

```

.hero-imagen img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  transition: transform 0.5s ease;
}

.hero:hover .hero-imagen img {
  transform: scale(1.05);
}

/* === SECCIONES === */
.seccion-titulo {
  font-family: var(--fuente-titulos);
  font-size: 32px;
  color: var(--color-secundario);
  text-align: center;
  margin-bottom: 40px;
  position: relative;
}

.seccion-titulo::after {
  content: "";
  display: block;
  width: 60px;
  height: 3px;
  background-color: var(--color-primario);
  margin: 15px auto 0;
}

section {
  padding: 60px 30px;
  max-width: 1200px;
  margin: 0 auto;
}

/* === ARTÍCULOS DESTACADOS === */
.grid-articulos {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 25px;
}

.tarjeta-articulo {
  background: white;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

```



```

.tarjeta-articulo:hover {
  transform: translateY(-6px);
  box-shadow: 0 12px 30px rgba(0, 0, 0, 0.12);
}

.tarjeta-articulo img {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

.tarjeta-contenido {
  padding: 25px;
}

.tarjeta-categoria {
  display: inline-block;
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 2px;
  color: var(--color-primario);
  font-weight: 700;
  margin-bottom: 10px;
}

.tarjeta-contenido h3 {
  font-family: var(--fuente-titulos);
  font-size: 22px;
  margin-bottom: 12px;
  color: var(--color-secundario);
}

.tarjeta-contenido p {
  font-size: 14px;
  line-height: 1.6;
  color: #666;
  margin-bottom: 15px;
}

.enlace-articulo {
  color: var(--color-primario);
  font-weight: 700;
  font-size: 14px;
  position: relative;
}

.enlace-articulo::after {
  content: "";
  position: absolute;
  bottom: -2px;
  left: 0;
  width: 0;

```

```

    height: 2px;
    background-color: var(--color-primario);
    transition: width 0.3s ease;
}

.enlace-articulo:hover::after {
    width: 100%;
}

/* === GALERÍA === */
.galeria {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 15px;
}

.galeria-item {
    overflow: hidden;
    border-radius: 8px;
}

.galeria-item img {
    width: 100%;
    height: 250px;
    object-fit: cover;
    transition: transform 0.4s ease;
}

.galeria-item:hover img {
    transform: scale(1.1);
}

/* === EVENTOS === */
.lista-eventos {
    display: grid;
    gap: 15px;
}

.tarjeta-evento {
    display: flex;
    align-items: center;
    gap: 25px;
    background: white;
    padding: 20px 25px;
    border-radius: 10px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.06);
    transition: transform 0.3s ease;
}

.tarjeta-evento:hover {
    transform: translateX(8px);
}

```

```

.evento-fecha {
  text-align: center;
  background-color: var(--color-primario);
  color: white;
  padding: 12px 18px;
  border-radius: 8px;
  min-width: 70px;
}

.evento-dia {
  display: block;
  font-size: 28px;
  font-weight: 900;
  font-family: var(--fuente-titulos);
  line-height: 1;
}

.evento-mes {
  display: block;
  font-size: 12px;
  text-transform: uppercase;
  letter-spacing: 1px;
  margin-top: 5px;
}

.evento-info h3 {
  font-family: var(--fuente-titulos);
  font-size: 18px;
  margin-bottom: 5px;
}

.evento-info p {
  font-size: 14px;
  color: #888;
}

/* === SUSCRIPCIÓN === */
.suscripcion-seccion {
  background: linear-gradient(135deg, var(--color-secundario), #16213e);
  color: white;
  border-radius: 15px;
  text-align: center;
  margin: 60px auto;
}

.suscripcion-contenido h2 {
  font-family: var(--fuente-titulos);
  font-size: 32px;
  margin-bottom: 15px;
}

```

```

.suscripcion-contenido p {
  font-size: 16px;
  color: rgba(255, 255, 255, 0.7);
  margin-bottom: 30px;
  max-width: 500px;
  margin-left: auto;
  margin-right: auto;
}

.suscripcion-form {
  display: flex;
  gap: 10px;
  max-width: 500px;
  margin: 0 auto;
}

.campo-formulario {
  flex: 1;
  padding: 14px 20px;
  border: none;
  border-radius: 6px;
  font-size: 16px;
  transition: box-shadow 0.3s ease;
}

.campo-formulario:focus {
  outline: none;
  box-shadow: 0 0 0 3px rgba(139, 0, 0, 0.3);
}

.btn-suscripcion {
  background-color: var(--color-primario);
  color: white;
  border: none;
  padding: 14px 28px;
  border-radius: 6px;
  font-weight: 700;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

.btn-suscripcion:hover {
  background-color: #a52a2a;
}

/* === FOOTER === */
.footer-principal {
  background-color: var(--color-secundario);
  color: white;
  padding: 60px 30px 0;
  margin-top: 60px;
}

```

```

.footer-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 30px;
  max-width: 1200px;
  margin: 0 auto;
}

.footer-grid h4 {
  font-family: var(--fuente-titulos);
  margin-bottom: 20px;
  color: var(--color-claro);
}

.footer-grid a {
  display: block;
  color: rgba(255, 255, 255, 0.6);
  margin-bottom: 10px;
  transition: color 0.3s ease;
}

.footer-grid a:hover {
  color: white;
}

.footer-bottom {
  text-align: center;
  padding: 30px 0;
  margin-top: 40px;
  border-top: 1px solid rgba(255, 255, 255, 0.1);
  font-size: 13px;
  color: rgba(255, 255, 255, 0.4);
}

/* === ANIMACIONES === */
@keyframes fadeInUp {
  from {
    opacity: 0;
    transform: translateY(30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* === RESPONSIVE === */
@media (max-width: 768px) {
  .header-principal nav {
    display: none;
  }
}

```

```

.hero {
  grid-template-columns: 1fr;
}

.hero-contenido {
  padding: 40px 25px;
}

.hero-titulo {
  font-size: 28px;
}

.grid-articulos {
  grid-template-columns: 1fr;
}

.galeria {
  grid-template-columns: repeat(2, 1fr);
}

.footer-grid {
  grid-template-columns: 1fr 1fr;
}

section {
  padding: 40px 20px;
}

.suscripcion-form {
  flex-direction: column;
}
}

@media (max-width: 480px) {
  .galeria {
    grid-template-columns: 1fr;
  }

  .footer-grid {
    grid-template-columns: 1fr;
  }

  .hero-titulo {
    font-size: 24px;
  }

  .seccion-titulo {
    font-size: 24px;
  }
}
}

```

Mateo guardó el archivo, abrió el navegador y recargó la página.

La portada apareció completa. El hero con dos columnas mostraba el título majestuoso y la imagen vibrante. Los artículos se alineaban en una cuadrícula perfecta. La galería desplegaba las obras de arte con elegancia. Los eventos se deslizaban uno tras otro. El formulario de suscripción invitaba a participar. El footer cerraba el diseño con solidez.

Redimensionó la ventana. Todo se adaptaba. Probó en el teléfono. Funcionaba.

Mateo se reclinó en su silla. Dos meses atrás no sabía ni qué era un selector. Ahora tenía un sitio web completo, responsivo, animado, diseñado profesionalmente.

—¿Listo para el lanzamiento? —preguntó Carolina, asomándose por la puerta.

—Listo —respondió Mateo.

—Entonces vamos. El sitio de "Cultura Viva" se publica hoy.

Mateo sonrió. No solo había aprendido CSS. Había descubierto que el diseño no era decoración: era la forma en que la tecnología se comunicaba con los humanos. Y él, Mateo Vargas, era ahora un puente entre ambos mundos.

Enigmas del Capítulo 10

Enigma 10.1: Modifica el CSS de la portada para que el header se vuelva translúcido (fondo semitransparente) cuando el usuario hace scroll. Pista: necesitarás una clase adicional (que se agregaría con JavaScript) para activar este efecto.

Enigma 10.2: Agrega una animación de entrada para las tarjetas de artículo. Cada tarjeta debe aparecer con un retraso progresivo (la primera inmediata, la segunda 0.2s después, la tercera 0.4s después).

Enigma 10.3: Crea una versión del hero donde la imagen de fondo use ``background-image`` en lugar de un ``<img>``. Ajusta el CSS para que funcione igual.

Enigma 10.4: Agrega un efecto de "parallax" suave al fondo de la sección de suscripción usando ``background-attachment: fixed``.

Enigma 10.5: Piensa en tres mejoras que le harías al diseño de la portada. Descríbelas y escribe el CSS necesario para implementarlas.

Conclusión

Lo que aprendiste en este libro

Has recorrido un largo camino desde el Capítulo 1, donde Mateo escribía su primer selector CSS. Hoy, has completado un viaje que abarca desde los fundamentos hasta un proyecto integrador completo de una portada de revista digital.

Concepto	Capítulo	Aplicación en "Cultura Viva"
Selectores y propiedades de texto	Cap 1	Dar estilo a encabezados y párrafos
Colores, fondos y tipografía	Cap 2	Identidad visual y paleta de colores
Modelo de caja	Cap 3	Espaciado, bordes y distribución
Display y positioning	Cap 4	Header fijo, barra lateral sticky
Flexbox	Cap 5	Galería de arte y layouts flexibles
CSS Grid	Cap 6	Portada con layout bidimensional
Responsive Design	Cap 7	Adaptación a móvil, tablet y desktop
Transiciones y animaciones	Cap 8	Botones con movimiento y vida
Pseudo-clases y pseudo-elementos	Cap 9	Hover effects, tooltips, capitulares
Proyecto final	Cap 10	Portada completa de revista

¿Qué sigue?

Si te ha gustado este enfoque de aprender CSS a través de una historia, aquí tienes algunas sugerencias:

1. **Practica con proyectos reales.** Toma cualquier sitio web simple y trata de replicar su diseño con CSS. La práctica es la mejor maestra.

2. **Explora conceptos avanzados:**

- **CSS Custom Properties (variables)** — Ya usaste algunas en `:root`. Puedes ir mucho más allá.

- **CSS Architecture** — Metodologías como BEM, SMACSS, o ITCSS para organizar tu CSS en proyectos grandes.

- **Preprocessors** — SASS, SCSS, o LESS agregan funcionalidades como variables, mixins y anidamiento.

- **CSS Frameworks** — Tailwind CSS, Bootstrap, o Foundation te permiten crear diseños rápidamente.

3. **Aprende JavaScript.** CSS da estilo, pero JS da interactividad avanzada. Menús hamburguesa, modales, sliders, y efectos dinámicos necesitan JavaScript.

4. **Sigue la historia:** Mateo Vargas continúa su viaje en los libros siguientes de la serie:

- **JavaScript Básico: El Código que Cobra Vida** — Variables, funciones, DOM, eventos

- **HTML Básico: Los Cimientos de la Web** — Estructura semántica, formularios, multimedia

5. **Comparte tu aprendizaje.** La mejor forma de aprender es enseñar. Ayuda a alguien más a dar sus primeros pasos con CSS.

El mensaje final

CSS no es solo un lenguaje de diseño. Es el medio por el cual la web se vuelve bella, funcional y humana. Cada regla CSS es una decisión: cómo guiar la mirada del usuario, cómo hacer que la información sea legible, cómo crear una experiencia placentera.

El sitio de "Cultura Viva" está en línea. Mateo descubrió que el diseño no es decoración, es comunicación. Y ahora, ese conocimiento es tuyo.

"El diseño es el alma de todo lo creado por el ser humano."

— Carolina, mentora en PixelLab

Alex Goyzueta Delgado

Lima, 2026

Apéndice: Soluciones a los Enigmas

Capítulo 1: El Primer Estilo

Enigma 1.1:

```
a {
  color: #006400;
  text-decoration: none;
  font-weight: bold;
}
```

Enigma 1.2:

```
.autor {
  font-style: italic;
  color: #666;
  font-size: 14px;
  text-align: right;
}
```

Enigma 1.3:

Un selector de clase (`.clase`) puede usarse en múltiples elementos. Un selector de ID (`#id`) debe ser único en la página. Usa clases para estilos reutilizables (ej: `.destacado`, `.tarjeta`). Usa ID para elementos únicos (ej: `#logo`, `#menu-principal`).

Enigma 1.4:

```
#titulo {
  color: navy;
  text-align: center;
  font-size: 40px;
}

.subtitulo {
  color: gray;
  font-size: 20px;
  text-transform: uppercase;
}

.intro {
  background-color: #eef;
  padding: 20px;
}
```

Enigma 1.5:

"En cascada" significa que cuando hay múltiples reglas CSS que afectan al mismo elemento, se aplica un orden de prioridad: la regla más específica o la que está definida después prevalece. También implica que los estilos se heredan de elementos padres a hijos (como el `font-family` del `body`).

Capítulo 2: La Paleta de Colores

Enigma 2.1:

```
body {  
  background: linear-gradient(#f5f0eb, #e8d5c4);  
  min-height: 100vh;  
}
```

Enigma 2.2:

```
a {  
  color: #333;  
  transition: color 0.3s ease;  
}  
  
a:hover {  
  color: #8B0000;  
}
```

Enigma 2.3:

HTML (en el ``<head>`):

```
<link href="https://fonts.googleapis.com/css2?family=Merriweather:wght@300;400;700&family=Montserrat:wght@
```

CSS:

```
article, .articulo {  
  font-family: "Merriweather", Georgia, serif;  
}  
  
nav, .menu, .navegacion {  
  font-family: "Montserrat", Arial, sans-serif;  
}
```

Enigma 2.4:

- `#FFFFFF` `rgb(255, 255, 255)`
- `#000000` `rgb(0, 0, 0)`
- `#8B0000` `rgb(139, 0, 0)`
- `#2E8B57` `rgb(46, 139, 87)`

Enigma 2.5:

`background-size: cover` escala la imagen para que cubra todo el contenedor, manteniendo la proporción. Recorta los bordes si es necesario. Sin `cover`, la imagen se muestra en su tamaño original y puede repetirse, dejando espacios blancos o distorsionándose.

Capítulo 3: El Espacio entre Líneas

Enigma 3.1:

Con `content-box` (por defecto):

ancho total = $200 + 15 + 15 + 3 + 3 = 236\text{px}$

Con `border-box`:

ancho total = **200px** (el padding y borde se restan del contenido).

Enigma 3.2:

```
.tarjeta-perfil {
  width: 280px;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  margin-bottom: 25px;
  background-color: white;
}
```

Enigma 3.3:

Los márgenes colapsados ocurren cuando dos márgenes verticales se encuentran y se fusionan en uno solo (el mayor). Ejemplo: un `h1` con `margin-bottom: 30px` seguido de un `p` con `margin-top: 20px`. El espacio entre ellos será 30px, no 50px.

Enigma 3.4:

```
.boton {
  background-color: #8B0000;
  color: white;
  text-decoration: none;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

.boton:hover {
  background-color: #a52a2a;
}
```

Enigma 3.5:

- `block`: ocupa todo el ancho disponible, respeta width/height, genera saltos de línea. Ej: `

`, `

`, `

`.
- `inline`: ocupa solo el espacio necesario, ignora width/height, NO genera saltos. Ej: ``, ``, `**`.**
- `inline-block`: como inline, PERO respeta width, height, margin y padding. Ej: botones en una fila, tarjetas en grid.

Capítulo 4: Poniendo Orden

Enigma 4.1:

```
.btn-volver-arriba {
  position: fixed;
  bottom: 30px;
  right: 30px;
  z-index: 9999;
  background-color: #8B0000;
  color: white;
  width: 50px;
  height: 50px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  text-decoration: none;
  font-size: 24px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.2);
  cursor: pointer;
}
```

Enigma 4.2:

`position: absolute` se posiciona en relación al ancestro posicionado más cercano. `position: fixed` se posiciona en relación al viewport (ventana del navegador). Usa `absolute` para elementos dentro de un contenedor (ej: etiqueta sobre imagen). Usa `fixed` para elementos que siempre deben estar visibles (ej: header, botón flotante).

Enigma 4.3:

```
.contenedor-tarjeta {
  position: relative;
}

.etiqueta-nuevo {
  position: absolute;
  top: 10px;
  left: 10px;
  background-color: #8B0000;
  color: white;
  padding: 4px 12px;
  border-radius: 4px;
  font-size: 12px;
  font-weight: bold;
  text-transform: uppercase;
}
```

Enigma 4.4:

Cuando los hijos tienen `float`, el contenedor padre colapsa (altura 0) porque los hijos flotantes salen del flujo normal. `overflow: hidden` (o `auto`) hace que el contenedor reconozca la altura de los hijos flotantes y se expanda para contenerlos. También se puede usar el `clearfix`.

Enigma 4.5:

El contexto de apilamiento es un grupo de elementos que comparten el mismo espacio en el eje

Z. Se crea con `position` + `z-index`, `opacity < 1`, `transform`, etc. El `z-index` solo compara elementos dentro del mismo contexto. Si dos elementos están en diferentes contextos de apilamiento, sus `z-index` no se comparan directamente.

Capítulo 5: Flexibilidad

Enigma 5.1:

```
.contenedor {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 400px;  
  gap: 20px;  
}
```

Enigma 5.2:

```
.nav-barra {  
  display: flex;  
  align-items: center;  
  padding: 0 20px;  
}  
  
.nav-links {  
  display: flex;  
  gap: 20px;  
  margin: 0 auto;  
}  
  
.btn-suscribirse {  
  margin-left: auto;  
}
```

Enigma 5.3:

`justify-content` alinea en el **eje principal** (horizontal por defecto). `align-items` alinea en el **eje transversal** (vertical por defecto). Si `flex-direction` cambia a `column`, los ejes se intercambian.

Enigma 5.4:

```
.tarjeta { flex: 1; }  
.tarjeta.medio { flex: 2; }
```

Flex: 1 divide el espacio disponible en 4 partes iguales (1+2+1). La tarjeta del medio ocupa 2 partes (la mitad), las otras ocupan 1 parte cada una (un cuarto).

Enigma 5.5:

`flex-wrap: wrap` permite que los elementos flexibles salten a la siguiente línea si no caben en el ancho del contenedor. Es esencial para diseño responsivo porque evita que los elementos se encojan demasiado o se salgan del contenedor.

Capítulo 6: La Grilla Perfecta

Enigma 6.1:

```
.grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 15px;
}
```

Enigma 6.2:

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr 250px;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "menu main widget"
    "footer footer footer";
  min-height: 100vh;
  gap: 15px;
}

.header { grid-area: header; }
.menu { grid-area: menu; }
.main { grid-area: main; }
.widget { grid-area: widget; }
.footer { grid-area: footer; }
```

Enigma 6.3:

`repeat(auto-fit, minmax(200px, 1fr))` crea tantas columnas de mínimo 200px como quepan en el contenedor. Si el contenedor es de 450px, crea 2 columnas. Si es de 650px, crea 3. Es responsivo automático.

`repeat(3, 1fr)` siempre crea exactamente 3 columnas iguales, sin importar el tamaño del contenedor.

Enigma 6.4:

```
.galeria-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}

.galeria-grid .destacada {
  grid-column: span 2;
  grid-row: span 2;
}
```

Enigma 6.5:

Usa **Flexbox** cuando trabajes en una sola dirección: filas de tarjetas, barras de navegación, centrar elementos, alinear contenido en un eje.

Usa **Grid** cuando necesites control bidimensional (filas y columnas): layouts de página completa, galerías complejas, portadas de revistas, dashboards.

Capítulo 7: Adaptándose al Mundo

Enigma 7.1:

```
.nav-menu {
  display: flex;
  gap: 20px;
  list-style: none;
}

@media (max-width: 600px) {
  .nav-menu {
    flex-direction: column;
    gap: 10px;
  }
}
```

Enigma 7.2:

`em` es relativa al `font-size` del elemento padre. `rem` es relativa al `font-size` del elemento `` (raíz). Usa `rem` para tamaños globales (tipografía, espaciado) porque permite escalar todo el sitio cambiando un solo valor. Usa `em` para tamaños que deben escalar con su contenedor local.

Enigma 7.3:

```
.tarjeta {
  width: 300px;
  display: inline-block;
}

@media (max-width: 480px) {
  .tarjeta {
    width: 100%;
    display: block;
  }
}
```

Enigma 7.4:

La etiqueta `

Enigma 7.5:

```
.hero {
  height: 100vh;
  background-image: url("hero.jpg");
  background-size: cover;
  background-position: center;
```



```
display: flex;
align-items: center;
justify-content: center;
color: white;
text-align: center;
}
```

Capítulo 8: El Toque Mágico

Enigma 8.1:

```
@keyframes bounce {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
}

.elemento {
  animation: bounce 1s ease-in-out infinite;
}
```

Enigma 8.2:

```
.boton {
  background-color: #8B0000;
  color: white;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.3s ease, transform 0.2s ease, box-shadow 0.3s ease;
}

.boton:hover {
  background-color: #a52a2a;
  transform: scale(1.05);
  box-shadow: 0 6px 20px rgba(139, 0, 0, 0.3);
}
```

Enigma 8.3:

```
@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}

.spinner {
  width: 40px;
  height: 40px;
  border: 4px solid #ddd;
  border-top-color: #8B0000;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}
```

```
}
```

Enigma 8.4:

``transition`` anima cambios de un estado a otro (ej: hover a normal). Se activa automáticamente cuando cambia una propiedad. Ideal para hover effects y cambios simples.

``animation`` permite animaciones más complejas con múltiples pasos, repeticiones, y control independiente. Necesita ``@keyframes``. Ideal para animaciones continuas (spinners), secuencias, o animaciones de entrada.

Enigma 8.5:

``animation-fill-mode: forwards`` mantiene el estado final de la animación cuando esta termina. Sin esto, el elemento vuelve a su estado inicial. Es útil para animaciones de entrada (fadeIn, slideIn) donde el elemento debe quedarse visible después de la animación.

Capítulo 9: Detalles que Importan

Enigma 9.1:

```
.destacados li::before {  
  content: "";  
  color: gold;  
  margin-right: 8px;  
}
```

Enigma 9.2:

```
.nav-menu a {  
  position: relative;  
  text-decoration: none;  
  color: #333;  
}  
  
.nav-menu a::after {  
  content: "";  
  position: absolute;  
  bottom: -2px;  
  left: 0;  
  width: 0;  
  height: 2px;  
  background-color: #8B0000;  
  transition: width 0.3s ease;  
}  
  
.nav-menu a:hover::after {  
  width: 100%;  
}
```

Enigma 9.3:

```
.galeria-item:nth-child(3),  
.galeria-item:nth-child(6) {  
  border: 3px solid gold;  
}
```

```
border-radius: 8px;
}
```

Enigma 9.4:

```
[data-tooltip] {
  position: relative;
  cursor: help;
}

[data-tooltip]::after {
  content: attr(data-tooltip);
  position: absolute;
  bottom: 100%;
  left: 50%;
  transform: translateX(-50%);
  background: #333;
  color: white;
  padding: 6px 12px;
  border-radius: 4px;
  font-size: 13px;
  white-space: nowrap;
  opacity: 0;
  transition: opacity 0.3s ease;
  pointer-events: none;
}

[data-tooltip]:hover::after {
  opacity: 1;
}
```

Enigma 9.5:

Pseudo-clases (``) describen un estado del elemento: ``:hover`` (mouse encima), ``:focus`` (enfocado), ``:nth-child(2)`` (segundo hijo).

Pseudo-elementos (``::``) crean elementos virtuales: ``::before`` (contenido antes del elemento), ``::after`` (después), ``::first-letter`` (primera letra).

Capítulo 10: El Lanzamiento

Enigma 10.1:

```
.header-principal.translucido {
  background-color: rgba(26, 26, 46, 0.9);
  backdrop-filter: blur(10px);
}
```

(La clase ``.translucido`` se agregaría con JavaScript al hacer scroll)

Enigma 10.2:

```
.grid-articulos .tarjeta-articulo {
  animation: fadeInUp 0.6s ease-out both;
}
```

```
.grid-articulos .tarjeta-articulo:nth-child(1) { animation-delay: 0s; }
.grid-articulos .tarjeta-articulo:nth-child(2) { animation-delay: 0.2s; }
.grid-articulos .tarjeta-articulo:nth-child(3) { animation-delay: 0.4s; }
```

Enigma 10.3:

```
.hero {
  background-image: url("hero.jpg");
  background-size: cover;
  background-position: center;
  position: relative;
}

.hero::before {
  content: "";
  position: absolute;
  inset: 0;
  background: rgba(26, 26, 46, 0.6);
}

.hero-contenido {
  position: relative;
  z-index: 1;
}
```

Enigma 10.4:

```
.suscripcion-seccion {
  background-attachment: fixed;
}
```

Enigma 10.5:

Respuesta libre. Ejemplos de mejoras:

1. Menú hamburguesa en móvil con animación
2. Modo oscuro con variables CSS y `prefers-color-scheme`
3. Loader de página con animación de entrada
4. Efecto de scroll reveal con `Intersection Observer` + CSS

Nota: Estas soluciones son una guía. No hay una única forma correcta de resolver los enigmas. Si tu solución funciona y tiene sentido, ¡es válida!

Sobre el Autor

Alex Goyzueta Delgado

Analista de datos, escritor y divulgador de tecnología. Peruano, nacido en Ancon Lima.

Alex combina su formación en análisis de datos con una profunda pasión por la narrativa y la educación. Cree firmemente que la mejor forma de aprender herramientas técnicas es a través de historias que conecten con nuestras emociones y experiencias cotidianas.

Serie de Desarrollo Web:

- **HTML Básico: Los Cimientos de la Web** — Aprende HTML desde cero con proyectos reales

- **CSS Básico: El Diseño que Transforma la Web** — Diseño web, selectores, Flexbox, Grid, animaciones

- **JavaScript Básico: El Código que Cobra Vida** — Programación desde cero con JavaScript

Trilogía de Excel:

- **Excel Básico: El Legado de las Fórmulas** — Aprende Excel desde cero con Sofía Mendoza

- **Excel Intermedio: Los Secretos del Taller** — Funciones lógicas, BUSCARV, tablas dinámicas

- **Excel Avanzado: El Imperio de los Datos** — Macros, Power Query, dashboards profesionales

Contacto:

- Correo: alexgoyzueta2018@gmail.com
- Temas: Excel, análisis de datos, narrativa educativa, desarrollo web, productividad
- Repositorio: github.com/SistGoyPeru/Libros

"El conocimiento no se crea ni se destruye: se transforma. De madera a muebles. De números a historias. De datos a decisiones."